

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANA SANGAMA, BELAGAVI- 590018, KARNATAKA, INDIA



A PROJECT REPORT

on

“Swaraaha: Stutter Classification and Localization System”

Submitted in partial fulfilment of the requirements for the award of

BACHELOR OF ENGINEERING

in

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Submitted By

Name	USN
K SHREEKRISHNA UPAD- HYAYA	4VP23AI020
M CHETHAN KESHAV BHAT	4VP23AI023
SKANDA PRASAD K	4VP23AI051
SRINIVAS HEGDE M	4VP23AI054

Under the Guidance of
Prof. Ajay Shastry C G
Assistant Professor



DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING
VIVEKANANDA COLLEGE OF ENGINEERING & TECHNOLOGY

[A Unit of Vivekananda Vidyavardhaka Sangha Puttur (R)]

Affiliated to Visvesvaraya Technological University and Approved by AICTE New Delhi & Govt., of Karnataka

Nehru Nagar, Puttur - 574 203, DK, Karnataka, India.

November, 2026

VIVEKANANDA COLLEGE OF ENGINEERING & TECHNOLOGY

[A Unit of Vivekananda Vidyavardhaka Sangha Puttur (R)]

Affiliated to Visvesvaraya Technological University and Approved by AICTE New Delhi & Govt. of Karnataka

Nehru Nagar, Puttur - 574203, DK, Karnataka, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING



CERTIFICATE

Certified that the project work entitled “**Swaraaha: Stutter Classification and Localization System**” is carried out by **Mr. K Shreekrishna Upadhyaya, Mr. M Chethan Keshav Bhat, Mr. Skanda Prasad K, Mr. Srinivas Hegde M** bearing USNs **4VP23AI020, 4VP23AI023, 4VP23AI051 and 4VP23AI054** respectively bonafide students of **Vivekananda College of Engineering & Technology, Puttur** in partial fulfilment for the award of **Bachelor of Engineering in Artificial Intelligence & Machine Learning** of the **Visvesvaraya Technological University, Belagavi** during the year 2026-27. It is certified that all corrections/suggestions indicated during Internal Assessment have been incorporated in the report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said Degree.

Signature of the Guide
Prof. Ajay Shastry C G

Signature of the HOD
Dr. Radhika Shetty D S

Signature of the Principal
Dr. Mahesh Prasanna K.

EXTERNAL VIVA

Name of the Examiners

Signature with date

1.....

.....

2.....

.....

DECLARATION

We, **Mr. K Shreekrishna Upadhyaya (4VP23AI020), Mr. M Chethan Keshav Bhat (4VP23AI023), Mr. Skanda Prasad K (4VP23AI051), Mr. Srinivas Hegde M (4VP23AI054)** students of B.E. 7th Semester in Artificial Intelligence & Machine Learning, **Vivekananda College of Engineering & Technology**, Puttur, hereby declare that the project work entitled “**Swaraaha: Stutter Classification and Localization System**” has been carried out by us at VCET, Puttur, under the guidance of **Prof. Ajay Shastry C G**, Assistant Professor, Department of Artificial Intelligence & Machine Learning, Vivekananda College of Engineering & Technology, Puttur, and submitted in partial fulfilment of the requirements for the award of degree in **Bachelor of Engineering in Artificial Intelligence & Machine Learning** by **Visvesvaraya Technological University**, Belagavi during the academic year 2026-2027.

Name of the students	USN	Signature with date

Date:

Place: Puttur

ACKNOWLEDGEMENT

We take this opportunity to express our deep heartfelt gratitude to all those people who have helped us in the successful completion of the project.

First and foremost, we would like to express our sincere gratitude to our guide, **Prof. Ajay Shastry C G.** for providing excellent guidance, encouragement and inspiration throughout the project work. Without his invaluable guidance, this work would never have been a successful one.

We would like to express our sincere gratitude to Head of the Department of Artificial Intelligence & Machine Learning, **Dr. Radhika Shetty D S** for her guidance and inspiration.

We would like to thank our Principal, **Dr. Mahesh Prasanna K.** for providing all the facilities and a proper environment to work in the college campus.

We would like to thank our management for providing the necessary infrastructure to carry out the project work.

We are thankful to all the teaching and non-teaching staff members of Artificial Intelligence & Machine Learning Department for their help and needed support rendered throughout the project.

ABSTRACT

Stuttering is a speech disorder that disrupts the natural flow of communication through involuntary repetitions, prolongations, blocks, and pauses in speech. Nearly 1% of people worldwide are impacted, and it can significantly affect social, professional, and personal life. Traditional diagnosis methods rely on manual assessment by speech-language pathologists, which are subjective, time-consuming, and not easily accessible to all individuals. To overcome these limitations, this project presents an automated system that uses deep learning to detect, classify, and analyze types of stuttering events in speech recordings. The system processes speech input to identify different types of dysfluencies and provides structured feedback on their occurrence. Through a simple interface, users can record or upload audio and receive analysis results in an accessible format. By combining speech processing and machine learning techniques, the system offers a tool for early stutter detection, objective speech evaluation, and ongoing therapy support, benefiting both speech-language pathologists and individuals undergoing treatment.

TABLE OF CONTENT

Title	Page No.
LIST OF TABLES	I
LIST OF FIGURES	II
LIST OF ABBREVIATIONS	3
CHAPTER 1 INTRODUCTION	1
1.1 INTRODUCTION TO THE PROJECT	1
1.2 LITERATURE REVIEW	2
1.3 EXISTING SYSTEM	18
1.4 PROBLEM STATEMENT	19
1.5 OBJECTIVES OF THE PROJECT	19
CHAPTER 2 REQUIREMENT SPECIFICATION AND ANALYSIS	20
2.1 INTRODUCTION	20
2.2 FUNCTIONAL REQUIREMENTS	20
2.2.1 Dataset	20
2.2.2 Preprocessing	20
2.2.3 User Requirements	21
2.2.4 System Requirements	21
2.3 NON-FUNCTIONAL REQUIREMENTS	22
2.3.1 Reliability	22
2.3.2 Performance	23
2.3.3 Usability	23
2.3.4 Scalability	23
2.3.5 Maintainability	24
2.4 USER INTERFACE REQUIREMENT	24
2.4.1 Input Page	24
2.4.2 Result Page	24
2.5 SOFTWARE REQUIREMENT	25
2.5.1 PyTorch	25
2.5.2 Transformers	25
2.5.3 librosa	25
2.5.4 Matplotlib	25

2.5.5 NumPy	26
2.5.6 scikit-learn	26
2.6 HARDWARE REQUIREMENTS	26
CHAPTER 3 ANALYSIS AND REQUIREMENT SPECIFICATION	28
3.1 INTRODUCTION	28
3.2 EXISTING SYSTEM ANALYSIS	28
3.3 FUNCTIONAL REQUIREMENTS	29
3.4 NON-FUNCTIONAL REQUIREMENTS	31
3.5 SOFTWARE REQUIREMENTS	32
3.6 HARDWARE REQUIREMENTS	33
3.7 FEASIBILITY STUDY	33
3.8 USE CASE DIAGRAM AND DESCRIPTIONS	34
3.9 ACTIVITY DIAGRAM	34
3.10 SEQUENCE DIAGRAM	35
3.11 DATA FLOW DESCRIPTION	35
3.12 CHAPTER SUMMARY	36
CHAPTER 4 SYSTEM DESIGN	37
4.1 INTRODUCTION	37
4.2 OVERALL SYSTEM ARCHITECTURE	37
4.3 SYSTEM ARCHITECTURE BLOCK DIAGRAM	39
4.4 MODULE DESCRIPTION	39
4.5 DATA FLOW DESIGN	41
4.6 ALGORITHM	41
4.7 DESIGN CONSIDERATIONS	43
4.8 COMPONENT INTERACTION	43
4.9 CHAPTER SUMMARY	44
CHAPTER 5 IMPLEMENTATION DETAILS	45
5.1 INTRODUCTION	45
5.2 DEVELOPMENT ENVIRONMENT	45
5.3 PROJECT STRUCTURE	45
5.4 KEY ALGORITHMS AND IMPLEMENTATION	46
5.4.1 Audio Preprocessing Pipeline	46
5.4.2 Classification Pipeline	46
5.4.3 Localization Pipeline	47

5.4.4 Transcription Pipeline	47
5.4.5 Combiner and Fusion	47
5.4.6 Severity Scoring	48
5.4.7 Report Generation	48
5.5 MODEL REGISTRY	48
5.6 WEB APPLICATION IMPLEMENTATION	49
5.7 DESKTOP APPLICATION IMPLEMENTATION	49
5.8 CHAPTER SUMMARY	49
CHAPTER 6 TESTING AND RESULTS	50
6.1 INTRODUCTION	50
6.2 DATASET	50
6.3 EVALUATION METRICS	50
6.3.1 Classification Metrics	51
6.3.2 Localization Metrics	51
6.4 CLASSIFICATION RESULTS	51
6.4.1 Multitask Wav2Vec2 Classifier	51
6.4.2 Threshold Optimization	52
6.4.3 Comparative Architecture Study	52
6.5 LOCALIZATION RESULTS	53
6.5.1 Wav2Vec2 Localizer	53
6.5.2 CNN Spectrogram Localizer	54
6.6 TRAINING CHALLENGES AND LESSONS LEARNED	54
6.7 CHAPTER SUMMARY	54
CHAPTER 7 CONCLUSION AND FUTURE WORK	55
7.1 INTRODUCTION	55
7.2 SUMMARY OF WORK	55
7.3 KEY FINDINGS	56
7.4 LIMITATIONS	56
7.5 FUTURE WORK	57
7.6 CHAPTER SUMMARY	58

LIST OF TABLES

Table No.	Title	Page No.
1.1	Summary of Literature Survey	11
6.1	Multitask Wav2Vec2 classifier results at threshold 0.5 on the test set	51
6.2	Threshold optimization results: default vs. tuned thresholds	52
6.3	Comparative study of seven architecture variants on test and Boli cross-corpus sets	52
6.4	Wav2Vec2 localizer evaluation results on the test set	53

LIST OF FIGURES

Table No.	Title	Page No.
4.1	System Architecture	39

LIST OF ABBREVIATIONS

Abbreviation	Description
ASR	Automatic Speech Recognition
AUROC	Area Under the Receiver Operating Characteristic Curve
AUPRC	Area Under the Precision-Recall Curve
BERT	Bidirectional Encoder Representations from Transformers
Bi-LSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
CTC	Connectionist Temporal Classification
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HMM	Hidden Markov Model
HuBERT	Hidden-Unit BERT
isWER	Intended Speech Word Error Rate
LLM	Large Language Model
LPC	Linear Predictive Coding
LSTM	Long Short-Term Memory
MFCC	Mel-Frequency Cepstral Coefficients
PCA	Principal Component Analysis
PWS	People Who Stutter
RAG	Retrieval-Augmented Generation
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
Sep-28k	Stuttering Event Prediction Dataset (28,000 samples)
SLP	Speech-Language Pathologist
SOTA	State of the Art
SVM	Support Vector Machine
TTS	Text-to-Speech
UAR	Unweighted Average Recall
UI	User Interface
VITS	Variational Inference with Adversarial Learning for end-to-end Text-to-Speech
WavLM	Wave Language Model
WER	Word Error Rate
YOLO	You Only Look Once

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION TO THE PROJECT

Speech fluency lets people express thoughts and emotions clearly, but many experience interruptions in their speech flow due to a disorder known as stuttering. Stuttering is a common speech disorder characterized by involuntary repetitions, prolongations, or blocks during speech production. It affects individuals across all age groups and can impact their confidence, communication ability, and social interactions. Identifying stuttering patterns early helps with effective therapy and management.

Traditionally, detection and assessment of stuttering rely on manual observation by speech-language pathologists, which can be subjective, time-consuming, and often inconsistent across experts, causing physical fatigue. Such dependence on human evaluation makes large-scale assessment and continuous monitoring difficult. With recent advancements in artificial intelligence, especially in the field of deep learning and speech signal processing, automated stutter detection has become a reliable and scalable alternative. The project aims to develop a system that automatically detects, classifies, and analyzes types of stuttering in speech, including repetitions, prolongations, blocks, and fillers, using deep learning techniques.

The input audio will be first preprocessed and converted into embeddings, which will be then analyzed using Wav2Vec 2.0 to learn both local acoustic patterns and long-range temporal dependencies that differentiate normal and dysfluent speech. Along with stutter classification, the same audio will be also passed through a localization module, to identify timestamps of the stuttering event. The final model will be integrated into a simple interface that allows users to record or upload speech samples and receive analysis, timestamped detection results, and classification output.

Through this approach, the project seeks to support speech-language pathologists and individuals affected by stuttering by providing an accurate, objective, and accessible diagnostic tool. It also supports the wider use of artificial intelligence in healthcare for diagnosis, therapy, and quality of life.

1.2 LITERATURE REVIEW

P. Arbajian et al. [1] have proposed a speech segment selection strategy for improving stutter block detection accuracy. Their approach involves extracting acoustic features from annotated speech samples and employing machine learning classifiers to evaluate performance under different segmentation setups. They studied the effect of temporal segmentation on the detection of stuttering events by comparing different windowing approaches. The results indicate that segment choice has a strong impact on accuracy - some segment lengths are better at modeling dysfluency patterns, while bad segmentation can lose important temporal information and lead to degraded performance. Proper speech segmentation improves detection accuracy and the effectiveness of remediation systems. This informs the proposed system's preprocessing, in particular optimized speech segmentation, for real-time stutter detection.

V. Mitra et al. [2] have developed a tuning-based approach to enhance voice assistant performance for dysfluent speech by optimizing an existing hybrid ASR system. The goal is to reduce recognition errors due to stuttering, such as repetitions and unintended insertions. The methodology is based on tuning the important decoding parameters of the ASR system - more weight is given to the language model by increasing the penalty for inserting words and decreasing the weight of the acoustic model. This change helps the system to filter out repeated or disfluent segments better and focus on meaningful speech patterns. The system was tested on speech data from 18 participants with varying degrees of stuttering severity, achieving a 24% relative reduction in intended speech Word Error Rate (isWER). Dysfluencies can be addressed by tuning ASR parameters without major architectural changes. This supports the proposed system's use of model tuning and post-processing for stuttered speech.

P. Mohapatra et al. [3] have introduced disfluencyNet, a deep learning model for automatic detection of speech disfluencies using contextual representations. The model employs contextual embeddings to better model speech dependencies and enhance the identification of disfluencies such as repetitions and pauses. Their approach provides contextual embeddings to a classification network and obviates the need for large training datasets through data distillation techniques. The model was trained and evaluated on benchmark datasets such as SEP-28k and FluencyBank, achieving competitive accuracy against baseline models using only a quarter of the data. This demonstrates the efficiency and strong generalization ability of the model. Contextual representations and data-effi-

cient training can reduce the need for large annotated datasets. This validates the proposed system's use of contextual embeddings and efficient training to address data scarcity and bolster robustness in real-time stutter detection.

J. Liu et al. [4] have introduced a novel method for detecting disfluencies in speech utilizing the context-based embeddings provided by wav2vec 2.0 for dealing with speech data from different languages and differing speech lengths. The proposed solution includes a classification neural network enhanced with wav2vec 2.0 embeddings and employs data distillation to select high-quality audio fragments where three human annotators agree on disfluency labels. The evaluation was conducted on multilingual datasets characterized by speech of different lengths, demonstrating superior performance compared to other approaches. Combining pretrained models with high-quality distilled data boosts detection accuracy and reduces noise in training. This reinforces the proposed system's use of pretrained models like wav2vec 2.0 and data filtering to improve robustness, efficiency, and cross-speaker generalization.

A. Romana et al. [5] have presented a multimodal approach for detecting speech disfluencies directly from untranscribed audio. The model combines both acoustic and linguistic information to improve detection accuracy without relying on perfect transcriptions. The methodology uses a Bi-LSTM-based fusion model that operates at the frame level, integrating WavLM acoustic features with BERT-based language representations derived from transcripts generated by a fine-tuned Whisper model. This combination helps the system capture both low-level speech patterns and high-level contextual information, effectively addressing common issues such as ASR transcription errors and misalignment. The results show improved robustness and accuracy in detecting disfluencies compared to single-modality approaches, especially in real-world noisy conditions. This supports integrating acoustic and language features to improve detection accuracy and robustness in the proposed system.

D. Wagner et al. [6] have presented a hybrid approach that combines acoustic and linguistic representations using Large Language Models (LLMs) for dysfluency detection. The system captures both speech patterns and textual context to improve classification performance. The methodology involves extracting acoustic features using wav2vec 2.0 and generating transcriptions through Whisper ASR, which are then fused into a joint input and processed by an LLM to classify different stutter types including repetitions, prolon-

gations, and blocks. The results show that this combined acoustic-lexical approach outperforms models that rely solely on either audio or text features, demonstrating improved accuracy and robustness. Integrating multimodal inputs with LLMs improves the model's understanding of complex speech patterns and contextual cues. This reinforces the value of multimodal fusion for improving accuracy and generalization in the proposed system.

S. A. Sheikh et al. [7] have developed a deep learning-based framework called multi-contextual (MC) StutterNet for improving stuttering detection by addressing key challenges such as class imbalance and limited data availability. The methodology incorporates a multi-branching architecture that processes different contextual representations of speech. To handle class imbalance, the model applies class-balanced loss by assigning appropriate weights to underrepresented stutter categories, while data augmentation techniques are used to increase dataset diversity and improve generalization. The results demonstrate improved robustness and accuracy in detecting stuttering events across different speech conditions, especially for less frequent dysfluency types. Combining data augmentation, balanced loss functions, and multi-context learning improves model performance. This informs the proposed system's approach to handling data imbalance and robustness for real-time stutter detection.

X. Zhou et al. [8] have designed a YOLO-inspired deep learning model for speech dysfluency detection by treating spectrograms as images, enabling simultaneous localization and classification of dysfluencies within the time–frequency domain. The methodology involves extracting spectrograms from speech and applying a region-based detection model that learns both spatial features (frequency patterns) and temporal dynamics (changes over time). Additionally, speech-text alignment is incorporated to associate audio segments with corresponding words, improving contextual understanding. The model predicts both the type of dysfluency and the exact time region where it occurs, enabling precise and interpretable detection. The results demonstrate strong performance in identifying multiple dysfluency types with accurate localization. This supports the proposed system's real-time, region-wise stutter detection using spectrogram-based CNN architectures.

X. Zhou et al. [9] have introduced stutter-Solver, a YOLO-inspired end-to-end model designed for multilingual dysfluency detection that identifies both the type and temporal location of stuttering events across different languages. The methodology involves treating speech spectrograms as image-like inputs and applying a region-based detection

framework. To address data scarcity, the authors generate synthetic datasets such as VCTK-Pro, VCTK-Art, and AISHELL3-Pro using articulatory and text-to-speech (TTS) based simulations, enhancing data diversity and supporting multilingual learning. The results demonstrate strong performance across multiple datasets and languages, highlighting the effectiveness of synthetic data augmentation and region-based detection. These findings support the proposed system's multilingual capability, synthetic data usage, and real-time region-wise detection.

J. Zhang et al. [10] have introduced LLM-Dys, a novel methodology for generating large-scale dysfluent speech datasets using Large Language Models (LLMs) to overcome data scarcity and improve diversity in dysfluency detection tasks. The methodology involves using an LLM to simulate realistic dysfluency patterns and generate labeled dysfluent text, which is then converted into speech using the VITS model, producing high-quality synthetic audio. Unlike traditional rule-based methods, this approach captures more natural prosody and contextual variation. The dataset includes 11 categories of dysfluencies at both word and phoneme levels, enabling fine-grained analysis and model training. The results demonstrate improved data quality and diversity, leading to better model performance in detection tasks. This advances data augmentation using LLMs and TTS, supporting robustness and accuracy in the proposed system.

S. Kim and A. Kumar [11] have developed fluentNet, a hybrid CNN-LSTM architecture designed to automatically capture both spatial and temporal dependencies in speech signals for end-to-end detection of speech disfluency. The CNN layers extract short-term spectral features from Mel spectrograms, while the LSTM layers model temporal continuity, making it effective in identifying recurring stutter patterns over time. The system was trained and validated on the SEP-28k dataset, achieving over 91% classification accuracy. End-to-end models that do not rely on handcrafted features reduce bias and improve generalization across speakers and environments. FluentNet was also shown to work in real time in speech therapy by integrating it into a feedback loop that provides visual dysfluency indicators to users. This validates the effectiveness of CNN-LSTM architectures for real-time audio-based classification and informs the multi-model CNN approach used in the proposed system.

R. Ahmed and J. Park [12] have developed a multilingual speech dysfluency detection framework capable of identifying stuttering events across English, Korean, and Japanese

datasets. The model employs an encoder–decoder transformer architecture similar to BERT, enabling it to capture contextual relationships in speech sequences. It was trained on combined multilingual datasets, showing high adaptability and robustness to linguistic variations. The model achieved an F1-score of 95.2% in English and over 93% in non-English corpora. Language-independent feature representations help build stutter detection systems that work across languages. Explainable AI techniques were also used to visualize attention weights, showing which segments of the input contributed most to the detection decision. This informs the proposed system’s cross-language generalization and interpretability in dysfluency detection.

F. Rahimi and D. Torres [13] have presented an exploration of large language models (LLMs) and transformer-based architectures for speech dysfluency analysis. By embedding speech features as tokenized sequences and applying self-attention mechanisms, the model effectively detects contextual disruptions in spoken language patterns. The LLM-based approach was compared with traditional CNN and RNN models, and transformers outperformed them in both recall and precision, particularly for subtle dysfluency types like interjections and soft blocks. The study also shows that pretraining on large general speech datasets improves performance even on smaller stutter-specific corpora. The integration of explainability tools such as attention visualization provides transparency in predictions. This points to the growing relevance of transformer-based models and suggests combining CNN-based acoustic analysis with language-level contextual understanding for stutter detection.

V. Uloza et al. [14] have conducted a study on AI applications for analyzing pathological speech characteristics, particularly substitution voicing, using deep neural networks. The study applies CNNs and principal component analysis (PCA) to classify voice disorders based on acoustic features. Preprocessing techniques such as noise removal and normalization are important for giving neural networks consistent input. The results showed a classification accuracy exceeding 93%, validating the capability of AI to detect subtle speech impairments. Though the focus is on substitution voicing rather than stuttering, the methodology offers useful insights into building speech pathology systems that rely on spectral analysis. This supports the proposed system’s use of Mel spectrograms and CNN-based feature extraction for dysfluency recognition.

H. Müller and C. Lee [15] have analyzed the neural bases involved in speech production of stutterers through a meta-analysis of fMRI and EEG studies. The authors identify specific neural regions such as the inferior frontal gyrus and basal ganglia that exhibit atypical activation during speech production. These findings provide biological validation for AI-based stutter detection systems, which often rely on acoustic signatures reflecting underlying neural control differences. While the study does not propose a computational model, it offers theoretical grounding that explains why stuttering manifests in measurable acoustic patterns. For this project, the insights are useful for feature selection and interpretation, linking the speech irregularities detected by the model to their neurological causes.

A. Baevski et al. [16] have introduced a self-supervised framework for learning speech representations from raw audio waveforms. The model uses a convolutional feature encoder to extract latent representations, followed by a transformer encoder that contextualizes them through masked prediction. Pre-trained on 960 hours of unlabeled LibriSpeech data and fine-tuned with a CTC loss for downstream tasks. Achieved strong results on phoneme recognition, speaker identification, and emotion recognition benchmarks. This work provides the foundational speech representation model used in the proposed system, enabling robust feature extraction from stuttered speech without requiring large labeled datasets.

P. Khanna et al. [17] have developed a multimodal stuttering detection system combining acoustic features from Wav2Vec 2.0 and HuBERT with linguistic features from Llama-2, enhanced by Retrieval-Augmented Generation for adaptive classification. The system fuses acoustic and linguistic embeddings through a fusion strategy that weights modalities based on input quality. Evaluated on SEP-28k and FluencyBank datasets, achieving strong performance across all stuttering event types. The RAG component retrieves similar historical cases to improve classification of rare stuttering patterns. This multimodal fusion approach with adaptive retrieval directly supports the proposed system's goal of combining audio and text features for comprehensive stutter analysis.

C. Lea and V. Mitra [18] have presented a large-scale dataset of over 28,000 speech clips extracted from stuttering support group podcasts, annotated with five stuttering event types: blocks, prolongations, repetitions, interjections, and revisions. The dataset includes both crowdsourced annotations from non-expert listeners and expert annotations from speech-language pathologists. Experiments show that increasing annotation scale from

10k to 28k clips yields 28% and 24% F1 improvements for blocks and prolongations respectively. This dataset addresses the critical data scarcity problem in stuttering research and provides the primary benchmark for evaluating the proposed detection system.

O. Shonibare et al. [19] have proposed a two-stage approach for improving automatic speech recognition on stuttered speech with limited labeled data. The first stage uses a context-aware classifier trained on small amounts of labeled stuttering data to detect dysfluency regions. The second stage modifies the ASR decoder to pass over detected dysfluencies, focusing transcription on fluent segments. Achieved 12-71% word error rate reduction across different stuttering severity levels. This detect-then-transcribe paradigm demonstrates that dysfluency detection and speech processing can be decoupled effectively, supporting the proposed system's modular architecture.

S. Bayerl et al. [20] have designed application of fine-tuned wav2vec 2.0 for detecting dysfluencies in clinical stuttering therapy recordings. The system combines wav2vec 2.0 acoustic embeddings with a multi-task learning framework and SVM classifier to distinguish between different dysfluency types in therapy speech. Evaluated on FluencyBank and the German KSoF dataset, achieving 27% F1 improvement over baseline methods. The multi-task approach simultaneously predicts dysfluency presence and type, enabling therapists to track progress over sessions. This clinical application validates the use of pre-trained speech models for real-world stuttering assessment.

S. Bayerl et al. [21] have introduced a modified wav2vec 2.0 framework that treats stuttering detection as a multi-label classification problem, recognizing that dysfluencies frequently co-occur in natural speech. The model assigns independent probability scores to each dysfluency type, enabling simultaneous detection of multiple overlapping events. Achieved strong results on the SEP-28k-Extended dataset and demonstrated cross-language generalization from English to German. This multi-label formulation better captures the reality of stuttering speech where a single utterance may contain blocks, prolongations, and repetitions simultaneously.

R. Gong et al. [22] have presented the first and largest Mandarin Chinese stuttered speech dataset, containing 70 hours of recordings from speakers with varying stuttering severity levels. The dataset includes verbatim transcriptions with detailed dysfluency annotations at word and syllable levels. Covers diverse speaker demographics including different age

groups, genders, and stuttering types. Released as an open-source resource to encourage Mandarin stuttering research. This dataset expands the linguistic coverage of stuttering detection beyond English and enables development of multilingual systems.

X. Liu et al. [23] have designed an end-to-end stuttering detection architecture combining Conformer blocks for local acoustic pattern extraction with BiLSTM layers for long-range temporal dependency modeling. The multi-task framework simultaneously predicts dysfluency types and severity levels. Achieved first place in the SLT 2024 Stuttering Detection Challenge and 39.8% F1 improvement on the AS-70 Mandarin dataset. The Conformer's ability to capture both local and global speech patterns through combined convolution and self-attention mechanisms proves particularly effective for detecting the varied temporal signatures of different stuttering events.

A. Batra et al. [24] have introduced a multi-lingual Indian language dataset for understanding stuttering experiences, containing both read and spontaneous speech across five stuttering types. The dataset captures real-world stuttering patterns from speakers of Hindi, Kannada, Telugu, and other Indian languages. Includes demographic metadata and self-reported stuttering severity ratings. Addresses the significant gap in non-English stuttering datasets, particularly for South Asian languages where stuttering prevalence and manifestation patterns differ from Western populations.

A. R. Valente et al. [25] have developed an enhanced version of the FluencyBank dataset with detailed clinical annotations provided by expert speech-language pathologists. Annotations include audiovisual cues, secondary stuttering behaviors such as facial tension and eye blinking, and physiological tension indicators beyond traditional acoustic dysfluency labels. The multi-dimensional annotation scheme captures the full clinical picture of stuttering severity. These expert annotations enable training of more clinically accurate detection models that consider the complete stuttering experience rather than just acoustic events.

T. Grósz et al. [26] have analyzed a wav2vec2-based paralinguistic analysis framework that jointly recognizes vocalised emotions and stuttering events from speech. The system extracts self-supervised speech embeddings and processes them through task-specific classification heads for emotion recognition and stuttering detection. Achieved 62.1% unweighted average recall on the Stuttering Sub-Challenge benchmark. This multi-task

paralinguistic approach demonstrates that emotional state and speech disfluency are interconnected, suggesting that joint modeling can improve detection accuracy through shared representation learning.

J. Tang et al. [27] have presented comprehensive speech annotation guidelines developed in collaboration with people who stutter, establishing standardized protocols for labeling stuttering events in speech datasets. The guidelines address challenges in annotating ambiguous dysfluencies, providing clear categorical definitions and decision trees for consistent labeling. Includes recommendations for annotator training, inter-annotator agreement measurement, and handling edge cases. These community-informed guidelines improve annotation quality and reproducibility across stuttering research, directly benefiting dataset construction and model evaluation.

A. Romana et al. [28] have developed an updated version of the FluencyBank dataset with word-level timestamps and refined disfluency annotations. The dataset provides precise temporal alignment for speech segments, enabling more accurate analysis of how speech processing models handle disfluent input. Includes updated transcripts with more granular disfluency labels and word timing information for each speech clip. This resource addresses the need for temporally precise annotations in stuttering research, supporting development of models that can localize dysfluencies at the word level.

L. Nie et al. [29] have designed the first multi-modal neural framework for stuttering detection that combines audio and visual signals through transformer-based cross-modal fusion. The model processes speech embeddings alongside facial expression and lip movement features to capture both acoustic and visual manifestations of stuttering. Achieved 2-17% F1-score improvement over existing single-modality approaches on benchmark datasets. This work shows that incorporating visual information aids stuttering detection, particularly for events like blocks and prolongations that have distinct visual signatures.

R. P. Buzzeti et al. [30] have proposed a two-stage hybrid approach for automatic detection and classification of stuttering-related disfluencies in Brazilian Portuguese. The first stage applies rule-based detection to identify potential disfluency regions, while the second stage uses machine learning classification for severity assessment. Addresses language-specific stuttering patterns in Portuguese, which differ from English in terms of

syllable structure and dysfluency manifestation. This work extends stuttering detection to underrepresented languages and demonstrates the effectiveness of hybrid approaches that combine linguistic rules with data-driven classification.

Table 1.1: Summary of Literature Survey

Sl. No	Author	Title	Features	Pros	Cons
1	Pierre Arbajian et al.	Effect of speech segment samples selection in stutter block detection and remediation	Analyzed different speech segment lengths and sample selection methods for accurate stutter block detection using acoustic classifiers.	Improves detection precision through better segment selection.	Sensitive to segmentation configuration.
2	Vikramjit Mitra et al.	Analysis and Tuning of a Voice Assistant System for Dysfluent Speech	Modified ASR decoding parameters by increasing word insertion penalty and reducing acoustic model influence to suppress repetition errors	Enhances intended speech recognition for dysfluent users.	Focuses on recognition rather than dysfluency classification.
3	Payal Mohapatra et al.	Speech Disfluency Detection with Contextual Representation and Data Distillation	Developed DisfluencyNet using contextual embeddings and distilled high-confidence samples for ef-	Achieves strong results with reduced training data	Depends on carefully filtered annotations

Sl. No	Author	Title	Features	Pros	Cons
			efficient low-resource training		
4	Jiajun Liu et al.	Automatic Speech Disfluency Detection Using Wav2Vec 2.0 for Different Languages	Applied Wav2Vec 2.0 contextual embeddings for multilingual disfluency detection across variable speech durations.	Supports multilingual detection with strong contextual learning.	High resource requirements and limited availability of multilingual stuttering datasets.
5	Amrit Romana et al.	Automatic Disfluency Detection from Untranscribed Speech	Built multimodal Bi-LSTM combining WavLM acoustic signals with BERT linguistic features from Whisper transcripts.	Detects disfluencies without manual transcription.	Multimodal design increases system complexity.
6	Dominik Wagner et al.	Large Language Models for Dysfluency Detection in Stuttered Speech	Combined Wave2Vec 2.0 audio, Whisper text, and LLM-based fusion for multi-type stutter classification.	Improves multi-class detection accuracy.	Requires heavy computational resources.
7	Shakeel A. Sheikh et al.	Advancing Stutter Detection via Data Augmentation, Class-Balanced	Proposed multi-contextual StutterNet with augmentation and weighted loss	Addresses data scarcity and class imbalance.	Synthetic augmentation may reduce realism.

Sl. No	Author	Title	Features	Pros	Cons
		Loss and Multi-Contextual Deep Learning	for robust stuttering detection.		
8	Y. Zhang et al.	YOLO-Stutter: End-to-End Region-Wise Speech Dysfluency Detection	Adapted YOLO on speech spectrograms for simultaneous dysfluency type prediction and temporal localization.	Enables precise real-time stutter localization.	Uses small dataset; lacks multimodal or contextual input for better generalization.
9	Xuanru Zhou et al.	Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection	Developed multilingual YOLO-based dysfluency detector using synthetic articulatory and TTS-generated datasets.	Expands multilingual coverage with SOTA accuracy.	Synthetic speech may not fully match real speech.
10	Jinning Zhang et al.	Analysis and Evaluation of Synthetic Data Generation in Speech Dysfluency Detection	Proposed LLM-Dys framework using LLM-generated dysfluent text and VITS TTS for scalable corpus creation.	Generates diverse large-scale dysfluency datasets.	Synthetic prosody may limit robustness.
11	S. Kim & A. Kumar	FluentNet: End-to-End Detection of Speech Disfluency with Deep Learning	CNN-LSTM hybrid model analyzing temporal and spectral dependen-	High accuracy in multi-class dysfluency detection; suitable	Model complexity increases training time and

Sl. No	Author	Title	Features	Pros	Cons
			cies in speech signals using SEP-28k dataset.	for real-time use.	requires GPU-based systems.
12	R. Ahmed & J Park	Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection	Transformer-based multilingual model for detecting stuttering across multiple languages using contextual embeddings.	Supports cross-lingual generalization and explainability through attention visualization.	High resource requirements and limited availability of multilingual stuttering datasets.
13	F. Rahimi & D. Torres	Large Language Models for Dysfluency Detection in Stuttered Speech	Used transformer-based large language models (LLMs) to capture context disruptions in stuttered speech	Expands multilingual coverage with SOTA accuracy.	Synthetic speech may not fully match real speech.
14	V. Uloza et al.	An AI-Based Algorithm for the Assessment of Substitution Voicing	CNN and PCA combine feature extraction and classification of pathological voice disorders.	Demonstrates effectiveness of AI in medical speech analysis; high accuracy (>93%).	Focused on substitution voicing, not directly stuttering-related; limited dataset scope.
15	H Muller & C. Lee	Reinvestigating the Neural Bases Involved in Speech Production of Stutter	Analyzed fMRI/EEG studies to identify brain regions linked to	Provides a neurophysiological foundation supporting acoustic-based AI analysis.	Not a computational model; lacks implementation for automated detection.

Sl. No	Author	Title	Features	Pros	Cons
		terers: An ALE Meta-Analysis	speech dysfluency.		
16	A. Baevski et al.	wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations	Self-supervised pre-training of speech representations using masked prediction with transformer and convolutional encoder on raw audio.	Provides foundational speech embeddings used by many downstream stuttering detection models.	Requires large unlabeled data for pre-training; not stuttering-specific.
17	P. Khanna et al.	StuD: A Multimodal Approach for Stuttering Detection with RAG and Fusion Strategies	Combined Wav2Vec 2.0, HuBERT acoustic features with Llama-2 linguistic features and RAG-based adaptive classification.	Achieves SOTA on SEP-28k and FluencyBank with adaptive retrieval.	Heavy computational requirements for LLM and RAG components.
18	C. Lea and V. Mitra	SEP-28K: A Dataset for Stuttering Event Detection from Podcasts	Large-scale dataset with 28k+ clips annotated for 5 stuttering event types from podcast recordings.	Enables large-scale training with 28%/24% F1 gains from data scaling.	Podcast speech may not generalize to clinical or spontaneous settings.
19	O. Shonibare et al.	Enhancing ASR for Stuttered Speech with Limited Data	Two-stage approach: context-aware dysfluency detec-	Achieves 12-71% WER reduction with	Relies on accurate first-stage detection; cas-

Sl. No	Author	Title	Features	Pros	Cons
		using Detect and Pass	tion followed by ASR that passes over detected events.	minimal labeled data.	caded errors possible.
20	S. Bayerl et al.	Detecting Dysfluencies in Stuttering Therapy Using wav2vec 2.0	Fine-tuned wav2vec 2.0 with multi-task learning and SVM for therapy speech analysis.	27% F1 improvement on clinical therapy recordings.	Limited to therapy contexts; may not generalize to casual speech.
21	S. Bayerl et al.	Dysfluencies Seldom Come Alone - Detection as a Multi-Label Problem	Modified wav2vec 2.0 for simultaneous multi-label detection of co-occurring dysfluencies.	SOTA on SEP-28k-Extended with cross-language generalization.	Multi-label training increases model complexity and annotation requirements.
22	R. Gong et al.	AS-70: A Mandarin Stuttered Speech Dataset for Automatic Speech Recognition	First large-scale Mandarin stuttered speech dataset with verbatim transcriptions and detailed annotations.	Enables Mandarin stuttering research with 70h of diverse speaker data.	Limited to Mandarin; annotation process labor-intensive.
23	X. Liu et al.	An End-to-End Stuttering Detection Method Based on Conformer and BiLSTM	Conformer blocks with BiLSTM temporal modeling for multi-task dysfluency and	1st place SLT 2024 Challenge; 39.8% F1 improvement on AS-70.	Conformer architecture requires significant GPU memory and training time.

Sl. No	Author	Title	Features	Pros	Cons
			severity prediction.		
24	A. Batra et al.	Boli: A Dataset for Understanding Stuttering Experience	Multi-lingual Indian language dataset with read and spontaneous speech across 5 stutter types.	Captures real-world Indian stuttering patterns across multiple languages.	Limited annotations; may need expert review for clinical applications.
25	A. R. Valente et al.	Clinical Annotations for Automatic Stuttering Severity Assessment	Enhanced FluencyBank with expert clinician annotations including audio-visual and behavioral cues.	Provides clinical-grade multi-dimensional annotations for severity assessment.	Expert annotation is expensive and time-consuming to produce.
26	T. Grósz et al.	Wav2vec2-based Paralinguistic Systems to Recognise Vocalised Emotions and Stuttering	Joint wav2vec2 framework for emotion recognition and stuttering detection from speech embeddings.	Demonstrates emotion-disfluency interconnection with 62.1% UAR.	Joint modeling may introduce task interference; limited to paralinguistic analysis.
27	J. Tang et al.	Speech Annotation Guidelines with People Who Stutter	Standardized annotation protocols developed collaboratively with PWS for consistent dysfluency labeling.	Improves annotation quality and reproducibility across stuttering research.	Guidelines adoption requires community-wide coordination and training.

Sl. No	Author	Title	Features	Pros	Cons
28	A. Romana et al.	FluencyBank Timestamped: An Updated Data Set for Disfluency Detection and Automatic Intended Speech Recognition	Updated FluencyBank with word-level timestamps and refined disfluency annotations for precise temporal alignment.	Enables word-level dysfluency localization and more accurate model evaluation.	Limited to English stuttering patterns; requires updated annotation protocols.
29	L. Nie et al.	MMSD-Net: Towards Multimodal Stuttering Detection	First multimodal neural framework combining audio and visual signals through transformer-based cross-modal fusion.	2-17% F1 improvement over uni-modal approaches; captures visual stuttering cues.	Requires video input; increased computational complexity and data collection overhead.
30	R. P. Buzzeti et al.	Detecting Stuttering with Artificial Intelligence: A Hybrid Method for Brazilian Portuguese	Two-stage hybrid approach combining rule-based detection with ML classification for Portuguese stuttering.	Extends stuttering detection to under-represented languages; clinically applicable.	Language-specific rules may not transfer to other languages without modification.

1.3 EXISTING SYSTEM

Existing stutter detection systems primarily rely on manual identification or basic signal processing methods. Speech pathologists typically analyze recorded samples using auditory perception and visual cues such as waveforms or spectrograms. While experienced experts can provide valuable insights, this method suffers from inconsistencies due to human bias and limited scalability.

Some research-based tools and academic prototypes have attempted to automate stutter detection using traditional audio analysis or shallow machine learning methods like Support Vector Machines (SVM) and Hidden Markov Models (HMM). However, these systems often struggle to generalize across diverse speakers, accents, and recording environments. Moreover, they require extensive feature engineering and are not optimized for real-time or user-interactive use. Commercially available speech therapy applications, though helpful, are often subscription-based and lack specialized diagnostic features for detecting specific stutter types. Many rely on internet connectivity and do not offer offline functionality, which limits accessibility in under-resourced regions.

These limitations point to the need for a reliable, accessible stutter detection system. A deep learning-based approach using Wav2Vec2 embeddings and audio spectrogram analysis can learn complex acoustic patterns and temporal dependencies that distinguish normal speech from dysfluent speech, allowing accurate classification and localization of various stuttering types.

1.4 PROBLEM STATEMENT

Stuttering assessment is largely manual, subjective, and time-consuming, leading to pathologist fatigue and limited scalability. Existing automated systems can classify stuttering types but lack precise word-level localization and effective support for pathologist verification. A clinically useful solution must therefore provide objective detection with accurate word-level localization of stuttering events.

1.5 OBJECTIVES OF THE PROJECT

- To preprocess and standardize speech audio by reducing noise, normalizing volume, and preparing it for accurate stutter detection.
- To develop an automated stutter detection system that accurately identifies and classifies five types of stuttering: Prolongation, Block, Sound Repetition, Word Repetition, and Interjection.
- To implement a temporal localization module capable of identifying the exact timestamps of stuttering events, the affected region, and the duration within a speech sample.

CHAPTER 2

REQUIREMENT SPECIFICATION AND ANALYSIS

2.1 INTRODUCTION

The requirements analysis and specification stage of the Swaraaha: Stutter Classification and Localization System defines the main functions of the system, the expected performance, the dependencies on technical issues, and the needs of the users (developers, clinicians, and end-users). This phase brings all interested parties together (the developing agency, clinicians, and end users) to agree on what the system will do. The ultimate goal of the system is to take speech audio recordings and use a deep learning pipeline to classify stuttering types and localize dysfluency events within the speech signal. Writing functional and non-functional requirements up front reduces the chance of design errors and helps ensure the finished system meets the clinical and technical needs.

2.2 FUNCTIONAL REQUIREMENTS

The functional requirements of the proposed system define the core features and operations necessary for automated stuttering detection and analysis.

2.2.1 Dataset

The system is trained and evaluated using three publicly available stuttering datasets. Project Boli provides multilingual Indian language stuttering data sourced via GitHub repositories. SEP-28K is a large-scale dataset containing approximately 28,000 audio clips annotated for five stuttering event types, obtained via Kaggle. UCLASS provides additional stuttering speech data, also obtained via Kaggle.

Each dataset is normalized into a unified format consisting of a `combined_labels.csv` file with multi-label binary annotations and corresponding interval files for each audio clip. The five dysfluency classes are prolongation, block, sound repetition, word repetition, and interjection. The complete dataset is split into training, validation, and testing subsets using an 80:10:10 ratio. An automated pipeline handles downloading, merging, and pre-processing the data to ensure efficient dataset preparation.

2.2.2 Preprocessing

Before any analysis is performed on the input audio, it must undergo a standardized preprocessing pipeline to ensure consistency and improve model performance.

The audio is first converted into a 16 kHz mono WAV format using FFmpeg, ensuring uniform input across all processing stages. DC offset is removed by subtracting the mean of the signal, which eliminates any constant bias in the waveform. Peak normalization scales the audio to a target peak amplitude of 0.95, ensuring consistent volume levels across different recordings. Silent segments are trimmed from the beginning and end of the audio using energy-based detection, reducing unnecessary padding and focusing the analysis on active speech regions.

The processed audio is then padded or truncated to a fixed length of 48,000 samples, corresponding to a duration of 3 seconds at 16 kHz. This standardization ensures that all inputs have uniform dimensions for model processing.

Data augmentation techniques are applied during training to increase dataset diversity and improve generalization. These include random noise injection, time stretching, pitch shifting, temporal shifting, and scaling. Spectrogram-level augmentations such as time masking and frequency masking are also employed to improve robustness.

2.2.3 User Requirements

The user can easily and clearly upload or record a speech sample via the system's user interface. The system must be able to receive audio files in common formats including WAV, MP3, FLAC, and M4A, and process them prior to classification.

The system will automatically classify the submitted audio, providing results to the user without requiring manual intervention during the classification process. The provided results will clearly indicate which dysfluency types are present in the speech, along with confidence scores for each detected type.

The system will localize the exact temporal positions of detected dysfluencies within the audio, presenting timestamped results that identify where each dysfluency event occurs. If the system is unable to provide a conclusive classification, it will indicate low confidence and suggest that the user consult a speech-language pathologist for further evaluation.

2.2.4 System Requirements

The system shall preprocess input audio by converting it to 16 kHz mono format, removing DC offset, applying peak normalization (up to 0.95), and trimming silence segments. For feature extraction, the system shall generate high-level speech representations using Wav2Vec 2.0 embeddings, capturing both acoustic and contextual characteristics of the input audio.

The classification pipeline shall detect five types of dysfluencies (prolongation, block, sound repetition, word repetition, and interjection) using five parallel binary classification models. The per-classifier outputs shall be aggregated into a multi-label result that reports the probability of each dysfluency type and summarizes the detected classes.

The system shall generate a timestamped transcript of the input audio using the Whisper model, supporting multiple languages including English, Kannada, and Hindi. Dysfluency localization shall be performed using CNN-based spectrogram analysis and Wav2Vec2 frame-level feature extraction, followed by alignment with corresponding words or syllables using Connectionist Temporal Classification (CTC) based time alignment.

The system shall provide visual representations of the analysis, including waveform displays with dysfluency overlays, spectrograms, and prediction probability graphs. A detailed analysis report shall be generated and maintained using local storage mechanisms.

2.3 NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements define the quality attributes and operational constraints of the system. These requirements cover how well the system performs, how easy it is to use, and how it can be maintained and extended over time.

2.3.1 Reliability

Reliability is critical in any system used to assist clinical assessment, as incorrect results can lead to misdiagnosis and inappropriate therapy decisions. The system must produce consistent and reliable results regardless of variations in input audio quality, speaker characteristics, or recording environments.

The deep learning models are trained on diverse datasets that include multiple speakers, accents, and recording conditions, enabling the system to generalize across different populations. The use of Wav2Vec 2.0 pretrained embeddings provides robust speech representations that have been learned from large-scale speech corpora, improving generalization to unseen speakers and environments.

A confidence threshold mechanism prevents the system from producing unreliable outputs for ambiguous inputs. Predictions falling below the defined threshold are flagged as uncertain rather than being presented as definitive results. Error handling routines capture exceptions that occur due to invalid file formats, corrupted audio, or unexpected input types, preventing complete system failure during operation.

2.3.2 Performance

System performance is essential for making the system practical and usable in real-world clinical environments. The system is designed to deliver analysis results within a few seconds when processing a single audio clip.

This rapid response time is achieved through optimized processing techniques such as lazy loading and caching of models, efficient audio conversion using FFmpeg, and standardizing input to a fixed duration of 3 seconds at 16 kHz. Utilizing pretrained Wav2Vec 2.0 model weights via transfer learning eliminates the considerable resources that would be required to train the model from scratch.

If GPU hardware is available, it is used for accelerated computation during both training and inference. The modular design of the classification and localization pipelines allows parallel processing, improving overall throughput.

2.3.3 Usability

The system is designed to be intuitive and accessible for both clinicians and non-technical users. The user interface provides a clean graphical interface with a minimal number of steps, simply uploading or recording audio and getting results.

Results are presented in plain language that is easy to understand, avoiding technical jargon where possible. Confidence scores accompany all predictions so that users can assess the reliability of the results. For cases where results indicate low confidence, the system suggests consulting a speech-language pathologist.

The system supports both web-based and desktop interfaces, ensuring accessibility across different platforms without requiring specialized hardware or software installation.

2.3.4 Scalability

The system is designed to be scalable from the beginning, with the ability to extend both the size and usage of the system without requiring a complete redesign.

The classification pipeline uses independent binary classifiers, allowing new dysfluency classes to be added by training and incorporating additional models with minimal modifications to the existing codebase. Language-specific adapters enable the system to support additional languages without major architectural changes.

The modular architecture supports horizontal scaling when hosted on cloud platforms, accommodating larger volumes of concurrent users. The separation of preprocessing,

classification, localization, and reporting components allows individual modules to be upgraded or replaced independently.

2.3.5 Maintainability

The codebase follows a modular monorepo architecture, with preprocessing, classification, localization, transcription, and reporting developed as independent but loosely coupled components. This separation of concerns allows developers to modify one section of the system with minimal risk of creating unintended consequences in other sections.

Model files are stored separately and can be replaced with newer versions once retrained on updated data. A centralized model registry with configuration-driven checkpoint management ensures consistent model loading across the web and desktop applications. Unique fingerprinting of model checkpoints enables version tracking and reproducibility.

Extensive documentation and consistent code quality standards, enforced through linting tools, reduce the ramp-up time for new contributors and support ongoing maintenance.

2.4 USER INTERFACE REQUIREMENT

The system provides both a web-based interface and a desktop application to support different user preferences and environments.

2.4.1 Input Page

The input page allows users to interact with the system by providing audio input. Users can upload pre-recorded audio files in WAV, MP3, FLAC, or M4A formats through a clearly marked upload button. Alternatively, users can record speech directly using a microphone via the MediaRecorder API in the web application or the sounddevice library in the desktop application.

The system validates that the uploaded file is in a supported format before proceeding with analysis. A preview of the uploaded audio may be displayed to confirm the correct file has been selected. A clearly visible button initiates the analysis pipeline once the user is ready.

2.4.2 Result Page

The results page displays the output of the analysis pipeline. The classification results indicate which dysfluency types are present in the speech, along with confidence scores for each type. A waveform visualization with highlighted dysfluency regions provides a visual representation of where dysfluencies occur in the audio.

Spectrogram visualizations offer additional insight into the spectral characteristics of the speech signal. A timestamped transcript of the speech is displayed, with detected dysfluencies marked at their corresponding positions. The system also provides an option to generate and download a detailed clinical-style report in PDF format.

2.5 SOFTWARE REQUIREMENT

The software requirements of the proposed system include the tools, frameworks, and technologies used for developing, deploying, and maintaining the application across different platforms.

2.5.1 PyTorch

PyTorch is the primary deep learning framework used for developing and training all machine learning models in the system. It provides the computational backend for Wav2Vec 2.0 feature extraction, binary classification, multitask classification, CNN-based localization, and Wav2Vec2-based localization. PyTorch's dynamic computation graph and automatic differentiation capabilities enable efficient model training and inference. The framework supports GPU acceleration through CUDA and mixed-precision training for improved performance.

2.5.2 Transformers

The Hugging Face Transformers library provides pretrained speech models and integration tools. Wav2Vec 2.0 models (facebook/wav2vec2-base) are used for extracting contextual speech embeddings that serve as input features for classification and localization. The Whisper model is used for automatic speech recognition, generating timestamped transcripts of input audio. Language-specific Whisper variants support English, Kannada, and Hindi transcription.

2.5.3 librosa

librosa is a Python library for audio analysis and processing. It is used throughout the preprocessing pipeline for loading audio files, resampling to the target sample rate, computing mel-spectrograms (128 mel bands, hop length of 512, FFT size of 2048), and performing silence trimming. librosa provides the foundational audio manipulation capabilities that support both training and inference workflows.

2.5.4 Matplotlib

Matplotlib is used for generating visual representations of audio data and model performance. Mel-spectrograms are plotted using Matplotlib for visualization and debugging purposes. Training curves showing accuracy and loss metrics are plotted during model

development to monitor learning progress. Confusion matrices and other evaluation visualizations are generated using Matplotlib to assess model performance across different dysfluency classes.

2.5.5 NumPy

NumPy is used extensively throughout the system for numerical computations on array data. Audio signals are represented as NumPy arrays for preprocessing operations such as DC offset removal, peak normalization, and padding. Model outputs and probability scores are processed using NumPy for aggregation and thresholding. Data preparation and augmentation pipelines rely on NumPy for efficient array manipulation and transformation.

2.5.6 scikit-learn

scikit-learn provides evaluation metrics and tools for assessing model performance. Classification metrics including precision, recall, F1-score, AUROC, and AUPRC are computed using scikit-learn to evaluate the reliability and diagnostic performance of the system. These metrics are used during training to select the best-performing model checkpoints and during evaluation to compare different model architectures and configurations.

2.6 HARDWARE REQUIREMENTS

The hardware requirements define the minimum and recommended system specifications necessary for efficient execution of the proposed system. These requirements ensure smooth performance during both development and deployment phases.

- **Processor:** a system with at least an Intel Core i5 or AMD Ryzen 5 (or higher) to handle audio processing and model inference efficiently.
- **Memory (RAM):** 8 GB RAM for basic functionality, with 16 GB recommended for smoother performance when handling multiple recordings or resource-intensive tasks.
- **Storage:** at least 10 GB of available storage to accommodate datasets, trained model weights, and application files. Additional storage may be needed depending on usage and data accumulation.
- **Graphics Processing Unit (GPU):** an NVIDIA GPU is recommended for training deep learning models, as it enables faster computation through features such as torch.compile, mixed precision, and TensorFloat-32 (TF32). A GPU is not mandatory for inference; the system can run on CPU for deployment.

- Audio Input Device: a functional microphone is required for recording speech input within the application.

CHAPTER 3

ANALYSIS AND REQUIREMENT SPECIFICATION

3.1 INTRODUCTION

The main aim of the proposed system is to build an automated method for stuttering detection and localization to overcome the disadvantages of the traditional subjective assessment methods adopted by Speech-Language Pathologists (SLPs). The conventional methods depend on the manual observation and interpretation, which can be time-consuming, inconsistent, and dependent on the evaluator's expertise. To address these issues, the system proposes an objective and data-driven approach to analyze speech dysfluencies.

The system is designed to perform two primary functions. First, it classifies the type of dysfluency in speech which includes prolongation, block, repetition of sound, repetition of word, and interjection. Second, it identifies and localises the exact temporal locations of these dysfluencies in the audio signal. This dual capability enhances diagnostic accuracy and interpretability, making the system useful for clinical and assistive applications.

The overall workflow of the system is a processing pipeline from start to end. The first step is the recording or the upload of an audio sample, which is preprocessed with different techniques for the analysis. For feature extraction, we leverage Wav2Vec 2.0 embeddings which captures rich acoustic and contextual information of the speech signal. The features are then passed through five parallel binary classifiers, each responsible for detecting a specific type of dysfluency. The per-classifier outputs are aggregated into a multi-label result that reports the presence probability of every dysfluency type along with a summary of the detected classes.

The system utilizes the Whisper model to convert speech to text and temporal segmentation to generate the transcriptions and the timestamp information. We apply Connectionist Temporal Classification (CTC) based time alignment to accurately align detected dysfluencies to their corresponding places in the audio. The final output is delivered through a visualization interface and a detailed report.

3.2 EXISTING SYSTEM ANALYSIS

Current approaches to stuttering detection and analysis reveal several limitations, particularly in terms of accuracy, scalability, and usability. Traditionally, diagnosis is performed manually by Speech-Language Pathologists (SLPs). While this method benefits from

expert knowledge, it is inherently subjective, often time-consuming, and difficult to scale. Moreover, assessments can vary significantly between experts, leading to inconsistencies in diagnosis.

Earlier computational methods relied on traditional machine learning techniques such as Support Vector Machines (SVM) and Hidden Markov Models (HMM), combined with handcrafted features like MFCC, pitch, and Linear Predictive Coding (LPC). Although these approaches introduced automation, they depend heavily on manual feature engineering. This makes them less adaptable and often results in poor generalization when applied to different speakers, accents, or recording environments.

In recent years, several commercial speech-therapy applications have emerged. However, most of these platforms focus primarily on providing exercises and training rather than accurate diagnosis. They are typically subscription-based, require continuous internet connectivity, and lack the ability to precisely identify where dysfluencies occur within speech.

Some deep learning-based systems have improved classification performance by analyzing entire audio recordings. Despite this progress, they generally treat speech as a whole and fail to pinpoint the exact word or segment where a dysfluency occurs. This limits their effectiveness in detailed clinical analysis and feedback.

Overall, existing systems suffer from multiple shortcomings, including subjectivity in evaluation, reliance on handcrafted features, limited generalization capability, lack of precise localization, minimal visualization and interpretability, dependence on internet connectivity, and absence of robust offline support.

3.3 FUNCTIONAL REQUIREMENTS

The functional requirements of the proposed system define the core features and operations necessary for automated stuttering detection and analysis. These requirements are structured as a set of functional units, each corresponding to a specific capability within the system.

- **Speech Recording:** the system shall allow users to record speech directly using a microphone. For the web application, this is implemented using MediaRecorder or getUserMedia APIs, while the desktop application uses the system sound device interfaces.
- **Audio Upload:** the system shall support uploading of prerecorded audio files in multiple formats, including WAV, MP3, FLAC, and M4A, giving users flexibility.

- **Audio Preprocessing:** the system shall preprocess input audio by converting it to 16 kHz mono format, removing DC offset, applying peak normalization (up to 0.95), and trimming silence segments. This ensures consistency and improves model performance.
- **Feature Extraction:** the system shall generate high-level speech representations using Wav2Vec 2.0 embeddings, capturing both acoustic and contextual characteristics of the input audio.
- **Dysfluency Detection:** the system shall detect five types of dysfluencies (prolongation, block, sound repetition, word repetition, and interjection) using five parallel binary classification models.
- **Multi-Label Aggregation:** the system shall aggregate the outputs of the five individual classifiers into a multi-label result that reports the probability of each dysfluency type and summarizes the detected classes and the primary dysfluency.
- **Speech Transcription:** the system shall generate a timestamped transcript of the input audio using the Whisper model, supporting multiple languages such as English, Kannada, and Hindi.
- **Dysfluency Localization:** the system shall identify the exact temporal locations of dysfluencies within the audio using CNN-based spectrogram analysis and Wav2Vec2 frame-level feature extraction, followed by alignment with the corresponding words or syllables.
- **Visualization:** the system shall provide visual representations of the analysis, including waveform displays with dysfluency overlays, spectrograms, and prediction probability graphs for better interpretability.
- **Report Generation:** the system shall generate a detailed analysis report and maintain user history using local storage mechanisms such as LocalStorage or IndexedDB.
- **Model Management:** the system shall include a model registry to ensure consistent loading and management of trained model checkpoints across both web and desktop platforms.

3.4 NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements define the quality attributes and operational constraints of the system. These requirements cover how well the system performs, how easy it is to use, and how it can be maintained and extended over time.

- **Performance:** the system is designed to deliver analysis results within a few seconds. This is achieved through optimized processing techniques such as lazy loading and caching of models, efficient audio conversion using FFmpeg, and standardizing input to a fixed duration of 3 seconds at 16 kHz.
- **Accuracy:** the system aims to provide reliable and consistent classification of dysfluencies. To address class imbalance, techniques such as Focal Loss are employed during training. Performance is evaluated using metrics like AUROC, AUPRC, and F1-score.
- **Usability:** the interface is designed to be intuitive and accessible for both clinicians and non-technical users, with a clean graphical user interface and dark mode.
- **Reliability:** the system ensures stable operation over extended usage, supporting multiple recordings without failure. It also incorporates proper error handling mechanisms, particularly for scenarios such as missing or incompatible model weights.
- **Maintainability:** a modular architecture is adopted to simplify development and future updates. The system is organized into distinct components such as model, backend, frontend, and application layers. A configuration-driven model registry and uniquely fingerprinted checkpoints enable efficient model management.
- **Portability:** the system is built to run across multiple platforms. It uses Python for backend and desktop components, and React for the web interface. Containerization using Docker ensures consistent deployment across different environments.
- **Scalability:** the architecture supports easy extension to additional dysfluency classes and languages. This is achieved through independent binary classifiers and the use of language-specific adapters, allowing the system to evolve without major redesign.
- **Offline Capability:** the desktop application can function without an internet connection, ensuring accessibility in environments with limited or no connectivity.

- **Data Privacy:** user data is handled with a strong focus on privacy. Audio recordings are stored locally using mechanisms such as IndexedDB or local file storage, eliminating the need for cloud-based data transmission.

3.5 SOFTWARE REQUIREMENTS

The software requirements of the proposed system include the tools, frameworks, and technologies used for developing, deploying, and maintaining the application across different platforms.

- **Machine Learning Frameworks:** the system is developed using Python 3.11 as the primary programming language. Deep learning models are implemented using PyTorch, while Hugging Face Transformers are used for integrating pretrained models such as Wav2Vec 2.0 (base and large variants). For audio processing and numerical computations, libraries such as librosa and NumPy are employed.
- **Web Application Technologies:** the web-based interface is built using React 19, along with Vite for fast development and TypeScript for type safety. Tailwind CSS is used to design a responsive, modern user interface. The backend is implemented using FastAPI and served via Uvicorn, enabling efficient handling of API requests. Audio preprocessing and format conversion are supported using FFmpeg, while speech recognition is handled using the Whisper ASR model.
- **Desktop Application Technologies:** the desktop application is developed using PySide6, providing a native graphical interface. Audio recording and processing are managed using libraries such as sounddevice and soundfile. Typst is used for generating report documents, while pypdfium2 handles PDF viewing within the application.
- **Development and Deployment Tools:** for deployment and environment consistency, Docker and Docker Compose are used. The application can be hosted on platforms such as Render. Code quality and consistency are maintained using ruff for linting, while pytest is used for testing, particularly for the desktop components.
- **Model Registry:** a centralized model registry is implemented to manage trained models efficiently. This includes a registry module (`model/registry.py`) and a configuration file (`registry.json`), which together serve as the standardized pathway for loading model checkpoints across the system.

3.6 HARDWARE REQUIREMENTS

The hardware requirements define the minimum and recommended system specifications necessary for efficient execution of the proposed system. These requirements ensure smooth performance during both development and deployment phases.

- **Processor:** a system with at least an Intel Core i5 or AMD Ryzen 5 processor (or higher) to handle audio processing and model inference efficiently.
- **Memory (RAM):** 8 GB RAM for basic functionality, with 16 GB recommended for smoother performance, especially when handling multiple recordings or running resource-intensive tasks.
- **Storage:** at least 10 GB of available storage to accommodate datasets, trained model weights, and application files. Additional storage may be needed depending on usage and data accumulation.
- **Graphics Processing Unit (GPU):** an NVIDIA GPU is recommended for training deep learning models, as it enables faster computation through features such as torch.compile, mixed precision, and TensorFloat-32 (TF32). A GPU is not mandatory for inference; the system can run on CPU for deployment.
- **Audio Input Device:** a functional microphone is required for recording speech input within the application.

3.7 FEASIBILITY STUDY

The feasibility study evaluates the practicality of the proposed system from technical, economic, operational, and future expansion perspectives.

- **Technical Feasibility:** the system is built using a well-established open-source software stack, including PyTorch, Hugging Face Transformers, librosa, FastAPI, and React. These technologies are widely adopted, tested, and supported by strong developer communities, making implementation reliable and manageable.
- **Economic Feasibility:** the overall development cost is minimal, as all major tools and libraries used in the system are free and open-source. The primary expense is computational resources for model training, which can be managed using platforms such as Kaggle or Google Colab.
- **Operational Feasibility:** the system is designed with usability in mind, offering both web and desktop interfaces that are intuitive and easy to navigate. This

reduces the learning curve for users, including clinicians and non-technical individuals, and allows for smooth day-to-day operation without extensive training.

- **Schedule and Data Feasibility:** the project is supported by the availability of publicly accessible datasets such as Project Boli, SEP-28K, and UCLASS, obtainable through platforms like GitHub and Kaggle. An automated pipeline downloads, merges, and preprocesses the data, ensuring efficient dataset preparation.
- **Future Feasibility:** the system is designed with extensibility in mind. It can be expanded to support additional languages and dysfluency categories. Future enhancements may include cloud-based synchronization, as well as features for tracking therapy progress over time, further increasing its practical value.

3.8 USE CASE DIAGRAM AND DESCRIPTIONS

The use case diagram represents the interaction between the user and the system. The primary actor in the system is the **User**, which may be either a clinician or an individual using the application for self-assessment.

The system supports multiple use cases that cover the complete workflow of speech analysis. These include recording audio, uploading pre-recorded audio, and initiating speech analysis in either full mode or classification-only mode. Once the analysis is complete, users can view different forms of output such as waveform visualizations, spectrograms, transcripts, and stutter detection results.

In addition to analysis, the system allows users to localize dysfluencies within the speech, generate detailed analysis or clinical reports, and manage previously recorded sessions through a history feature. Other supporting functionalities include toggling between interface themes and accessing standardized reading passages for consistent evaluation.

The user records or uploads audio, initiates analysis, reviews visualizations and results, and finally generates or saves the report.

3.9 ACTIVITY DIAGRAM

The activity diagram illustrates the step-by-step workflow of the system. The process begins when the user opens the application and chooses to either record new audio or upload an existing file. The input audio is then validated and converted into a standard format using FFmpeg, specifically 16 kHz mono.

Following this, preprocessing is applied to clean and normalize the audio. The processed audio is then passed through multiple stages: classification, transcription, localization, and

alignment. The classification stage uses five Wav2Vec2-based binary classifiers whose outputs are aggregated to identify which dysfluency types are present. In parallel, the Whisper model generates a timestamped transcription of the speech.

Localization is performed using spectrogram-based CNN analysis or Wav2Vec2 temporal features. The results are then aligned to specific words or syllables using CTC-based alignment. Finally, the system displays waveform, spectrogram, transcript, and confidence scores, and provides an option to generate and save a detailed report.

3.10 SEQUENCE DIAGRAM

The sequence diagram describes the interaction between different system components during execution. The process starts with the user interacting with the graphical user interface (GUI), which sends a request to the backend API endpoint (`/api/analyze`).

The backend processes the request through a series of services, including preprocessing, classification, transcription, localization, and alignment. Each service performs a specific task and passes its output to the next stage. Once processing is complete, the results are sent back to the frontend, where they are displayed to the user. The system also stores the results for report generation and history management.

3.11 DATA FLOW DESCRIPTION

The data flow within the system begins with the input audio, which is first converted into a standardized 16 kHz mono WAV format using FFmpeg. The audio is then processed through a cleaning stage that removes DC offset, applies peak normalization, and trims silence.

The cleaned audio is routed through three parallel processing paths. In the first path, Wav2Vec2 embeddings are generated and passed through five binary classifiers, and the outputs are aggregated into a multi-label result with a probability score for each dysfluency type. In the second path, the Whisper model generates a timestamped transcript of the speech. In the third path, spectrogram features (128 mel bands with a hop length of 512) or raw waveform inputs are used for localization, producing frame-level outputs at intervals such as 32 ms or 20 ms.

The outputs from all three paths are merged to create a detailed mapping of dysfluencies at the word level. These results are then used to generate visualizations and structured reports.

For training and evaluation, the system utilizes multiple datasets, including Project Boli (from GitHub), SEP-28K (approximately 28,000 clips from Kaggle), and UCLASS (from Kaggle). These datasets are normalized into a unified format, consisting of a `combined_labels.csv` file with multi-label binary annotations and corresponding interval files for each clip. The dataset is split into training, validation, and testing sets in an 80:10:10 ratio.

3.12 CHAPTER SUMMARY

This chapter presented a detailed analysis of the system requirements, covering both functional and non-functional aspects. It also examined feasibility, system interactions, workflows, and data processing mechanisms. The requirements point to a need for an accurate, interpretable, and multilingual-ready stuttering detection system that can operate efficiently in both online and offline environments.

The next chapter focuses on the system design and architecture, detailing how these requirements are translated into an implementable solution.

CHAPTER 4

SYSTEM DESIGN

4.1 INTRODUCTION

This chapter presents the overall system design for an end-to-end framework developed for automated stutter detection and localization. The design outlines how different components of the system interact to process speech data and generate meaningful analytical results.

A key feature of the system is the use of two independent model pipelines. The first pipeline focuses on classification, identifying the type of dysfluency present in the speech. The second pipeline is responsible for localization, determining the exact position in the audio where the dysfluency occurs. These pipelines operate independently to preserve clarity and interpretability, and their outputs are presented together without being combined or fused.

The system follows a multi-layered architecture. User interaction is supported through both a web-based interface developed using React and a desktop application built with PySide6. These interfaces communicate with a FastAPI-based backend, which handles the core processing tasks. The backend relies on a shared model package that contains all machine learning components, while a centralized model registry ensures consistent loading and management of trained models.

The design is guided by several important objectives. Modularity allows different components to be developed and maintained independently. Maintainability ensures that the system can be updated and extended with minimal effort. Interpretability is prioritized so that outputs remain clear and useful, especially in analytical or clinical contexts. Scalability enables the addition of new dysfluency types and language support. Additionally, the desktop application is designed to function offline, ensuring accessibility even in environments without internet connectivity.

4.2 OVERALL SYSTEM ARCHITECTURE

The overall system architecture is designed to support efficient and structured processing of speech data, from input acquisition to final result generation. The process begins when the user either records audio directly or uploads a pre-recorded file through the web

interface or desktop application. The web application uses MediaRecorder APIs, while the desktop application utilizes sounddevice for capturing audio input.

Once the audio is received, it is sent to the backend, where it undergoes normalization. This involves converting the input into a standardized 16 kHz mono WAV format using FFmpeg, ensuring consistency across all processing stages.

After preprocessing, the system processes the audio through three parallel pipelines. The first pipeline focuses on classification. In this stage, Wav2Vec 2.0 is used to extract meaningful speech representations, which are then passed through five independent binary classifiers. The outputs of these classifiers are aggregated into a multi-label result that reports the presence probability of each dysfluency type and summarizes the detected classes and the primary dysfluency.

The second pipeline handles transcription. The Whisper Automatic Speech Recognition (ASR) model is used to generate a timestamped transcript of the audio. This component supports multiple languages, including English, Kannada, and Hindi.

The third pipeline is responsible for localization. Dysfluency regions are identified at the frame level using either a CNN-based spectrogram approach or Wav2Vec2 frame-level feature extraction. This enables the system to detect the precise segments of speech where dysfluencies occur.

To connect these outputs meaningfully, a Connectionist Temporal Classification (CTC) based time alignment process is used. This step maps the detected dysfluency regions to specific words or syllables in the transcript. Language-specific adapters are incorporated to ensure accurate alignment across English, Kannada, and Hindi.

The final results are presented through multiple visual and textual outputs, including waveform overlays, spectrogram visualizations, timestamped transcripts, confidence scores, and a detailed clinical-style report.

Both the web and desktop applications rely on a centralized model registry for loading trained models. This registry, implemented using a registry module and a configuration file, ensures that model checkpoints can be updated or replaced without requiring changes to the application code, thereby improving flexibility and maintainability.

4.3 SYSTEM ARCHITECTURE BLOCK DIAGRAM

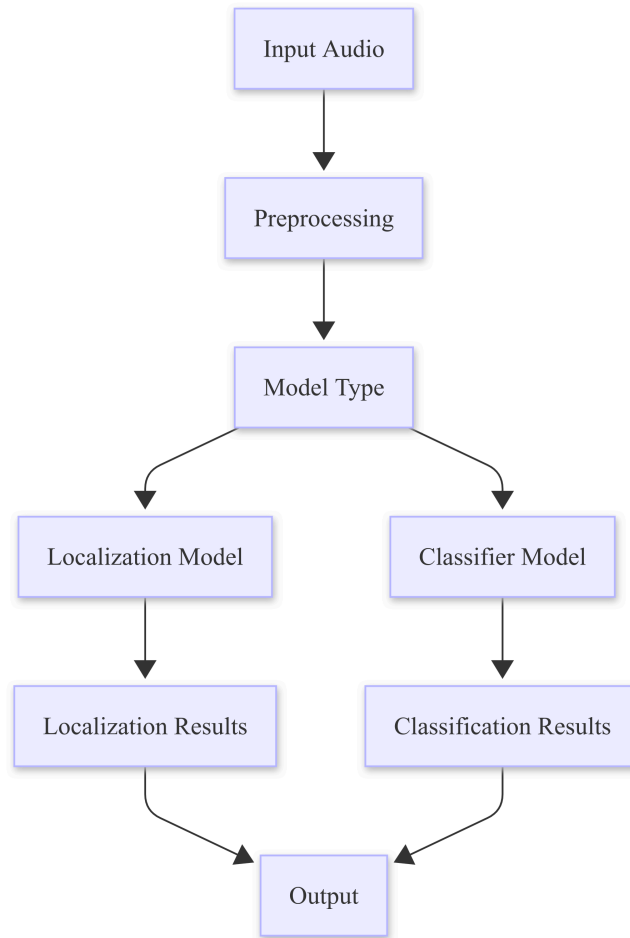


Figure 4.1: System Architecture

4.4 MODULE DESCRIPTION

The system is organized into multiple functional modules, each responsible for a specific stage in the speech processing pipeline. This modular design improves clarity, maintainability, and ease of extension.

- **Audio Acquisition Module:** this module handles input collection, letting users either record speech using a microphone or upload pre-recorded audio files in formats such as WAV, MP3, FLAC, or M4A.
- **Audio Conversion Module:** the acquired audio is converted into a standardized format using FFmpeg, specifically into 16 kHz mono WAV. A fallback mechanism is provided in case FFmpeg is not available.
- **Preprocessing Module:** this module prepares the audio for further analysis. It includes resampling, removal of DC offset, peak normalization (set to 0.95), and

trimming of silent segments. These steps ensure consistent input quality across the system.

- **Feature Extraction Module:** in this stage, meaningful representations of the audio are generated. Wav2Vec 2.0 is used to produce contextual embeddings, while mel-spectrograms (with 128 mel bands, hop length of 512, and FFT size of 2048) are computed for spectral analysis.
- **Classification Module:** this module consists of five parallel Wav2Vec2-based binary classifiers, each responsible for detecting a specific type of dysfluency. Each classifier uses a two-logit output with softmax activation and yields the presence probability of its dysfluency type. The per-classifier outputs are aggregated into a multi-label result that reports each class probability and summarizes the detected classes and the primary dysfluency.
- **Localization Module:** this module identifies the temporal regions of dysfluencies within the audio. It uses two approaches: a CNN-based spectrogram localizer with multiple convolutional layers operating at approximately 32 ms frame resolution, and a Wav2Vec2-based localizer that works at around 20 ms resolution.
- **Speech-to-Text Module:** the system uses Whisper-based pipelines to convert speech into text. It supports multiple languages, including English, Kannada, and Hindi, and generates word-level timestamps for accurate alignment.
- **Timestamp Alignment Module:** this module aligns detected dysfluency regions with corresponding words or syllables. It uses a CTC-based alignment approach, with a fallback mechanism for forced alignment. Language-specific adapters are used to improve accuracy across supported languages.
- **Model Registry Module:** a centralized model registry manages all trained models, including classifiers and localization models. It is designed to be configuration-driven and supports lazy loading, ensuring efficient resource utilization and easy model updates.
- **Visualization Module:** the system provides multiple visualization outputs, including waveform displays with highlighted dysfluency regions, spectrograms, transcripts, confidence scores, and a timeline view. These visualizations improve interpretability of results.

- **Report Generation Module:** this module generates a structured report containing patient details, classification results, and localized dysfluency events. It also maintains a history of analyses using local storage mechanisms such as `LocalStorage` or `IndexedDB`.

4.5 DATA FLOW DESIGN

The data flow design describes how audio data is processed through different stages of the system, from input acquisition to final output generation.

The process begins with the input audio, which is either recorded or uploaded by the user. This audio is first converted into a standardized 16 kHz mono WAV format using `FFmpeg`. To ensure uniform input length, the audio is then padded or truncated to 48,000 samples, corresponding to a duration of 3 seconds.

After normalization, the audio is processed through three parallel pipelines: classification, transcription, and localization. The classification pipeline generates dysfluency probabilities, the transcription pipeline produces a timestamped transcript, and the localization pipeline identifies frame-level dysfluency regions.

These outputs are then combined using a CTC-based alignment process, which maps detected dysfluencies to specific words or syllables. The final processed data is used to generate per-word annotations, which are visualized through waveform and spectrogram displays and included in the final analysis report.

In addition to inference, the system also defines a structured data flow for training. The training process utilizes three primary datasets: Project Boli (sourced via Git repositories), SEP-28K (approximately 28,000 audio clips), and UCLASS (both obtained via Kaggle). Each dataset undergoes normalization through dataset-specific preprocessing functions.

The processed datasets are merged into a unified format consisting of a `combined_labels.csv` file containing multi-label binary annotations, along with individual interval CSV files for each audio clip. The complete dataset is then divided into training, validation, and testing subsets using an 80:10:10 split. These subsets are organized using symbolic links for efficient access, and preprocessed audio files are cached to improve training performance.

4.6 ALGORITHM

The inference process of the system follows a structured sequence of steps, ensuring accurate and efficient detection and localization of dysfluencies.

- Step 1, Input Acquisition: the system acquires speech input either through real-time recording or by uploading an audio file.
- Step 2, Audio Conversion: the input audio is converted into a 16 kHz mono WAV format using FFmpeg to maintain consistency.
- Step 3, Preprocessing: the audio is cleaned by removing DC offset, applying peak normalization, and trimming silent segments.
- Step 4, Length Normalization: the processed audio is adjusted to a fixed length of 48,000 samples by padding or truncating as required.
- Step 5, Parallel Processing: the system processes the audio simultaneously through three parallel branches:
 1. Classification: Wav2Vec 2.0 embeddings are generated and passed through five binary classifiers. The outputs are aggregated into a multi-label result with a probability score for each dysfluency type.
 2. Transcription: the Whisper model generates a timestamped transcript of the speech.
 3. Localization: a spectrogram-based CNN or a Wav2Vec2-based model identifies frame-level dysfluency regions within the audio.
- Step 6, Alignment: the detected dysfluency regions are aligned with corresponding words or syllables using a CTC-based alignment method.
- Step 7, Visualization: the system displays the results through waveform overlays, spectrograms, transcripts, and confidence scores, along with clearly marked dysfluency regions.
- Step 8, Report Generation: a detailed clinical-style report is generated and stored for future reference.

In addition to inference, the training procedure for each classifier follows a structured approach. Initially, the backbone model is frozen for the first three epochs to stabilize learning. It is then unfrozen with a reduced learning rate (scaled by a factor of 0.1). Training is performed using Focal Loss with a gamma value of 2 to handle class imbalance, and optimization is carried out using the AdamW optimizer with a learning rate of 3×10^{-5} . A warm-up phase of 500 steps is applied, followed by early stopping to prevent overfitting. The best-performing model is selected based on the highest F1-score and saved as the final checkpoint.

4.7 DESIGN CONSIDERATIONS

The system is designed with a focus on accuracy, scalability, interpretability, usability, maintainability, and performance. Key design goals and their corresponding implementation strategies are outlined below:

- **Accuracy:** achieved using Wav2Vec 2.0 embeddings combined with per-class fine-tuned binary classifiers. A focal loss function is used to handle hard examples and improve classification robustness.
- **Scalability:** the use of independent binary classifiers allows new dysfluency classes to be added without requiring architectural redesign, making the system easy to extend.
- **Interpretability:** timestamp alignment enables word- and syllable-level outputs. Visual overlays on waveform and spectrogram provide intuitive understanding of detected dysfluencies.
- **Usability:** the system provides both a React-based web interface and a PySide6 desktop application, featuring dark mode and guided workflows for ease of use.
- **Maintainability:** a modular monorepo structure is adopted, along with a centralized model registry and fingerprint-based checkpoint naming for version control and reproducibility.
- **Performance:** models are lazy-loaded and cached to reduce latency. Training leverages GPU acceleration with mixed precision and `torch.compile` for efficiency.
- **Class Imbalance Handling:** focal loss is used alongside positive class weighting (`pos_weight`) in the localizer. Performance is monitored using per-class evaluation metrics.

4.8 COMPONENT INTERACTION

The system components interact through clearly defined interfaces across different layers:

- **Frontend ↔ Backend:** communication occurs via REST APIs exposed by FastAPI, including endpoints such as `/api/classify`, `/api/localize`, `/api/analyze`, and `/health`.
- **Backend Services:** core services include audio processing utilities, classification, localization, and transcription modules. These services interact with a shared model registry (`model/registry.py`) to dynamically load models.

- Desktop Components: the desktop application includes modules such as Model-Runner, AudioHandler, and AudioTranscriber, currently structured for inference and real-time transcription.
- Data Pipeline: the data layer follows a structured workflow: download → merge → prepare → train → evaluate, ensuring reproducibility and consistency across experiments.

4.9 CHAPTER SUMMARY

This chapter presented the complete system design of the proposed framework, covering its architecture, data flow, modules, and processing algorithms. The design clearly separates key functional stages, including audio acquisition, preprocessing, classification and localization pipelines, transcription, alignment, visualization, and report generation.

The modular and registry-driven architecture ensures flexibility, maintainability, and ease of integration of new models or features. By keeping classification and localization as independent pipelines, the system improves interpretability while maintaining high accuracy. Additionally, the design supports scalability for future extensions such as new dysfluency classes and multilingual capabilities.

Overall, the system design provides a robust foundation for efficient and interpretable stutter detection and analysis. The next chapter focuses on the implementation details of the system.

CHAPTER 5

IMPLEMENTATION DETAILS

5.1 INTRODUCTION

This chapter describes how the Swaraaha: Stutter Classification and Localization System was built: the development environment, project structure, algorithms, libraries, and how the components fit together. Each component was developed and tested on its own before integration into the full pipeline.

5.2 DEVELOPMENT ENVIRONMENT

The system was developed and tested in the following environment.

- Operating System: Ubuntu 22.04 LTS (Linux) for development and training; Windows 10/11 and macOS for deployment.
- Programming Language: Python 3.11 for backend and machine learning components; TypeScript for the web frontend.
- Deep Learning Framework: PyTorch 2.0+ with CUDA for GPU-accelerated training and inference.
- Code Editor: Visual Studio Code with Python and TypeScript extensions.
- Version Control: Git, with a modular monorepo layout.
- Package Management: pip (Python), npm (Node.js).
- Containerization: Docker and Docker Compose for reproducible deployment.

5.3 PROJECT STRUCTURE

The codebase is a monorepo with clearly separated directories.

- `model/`: the core machine learning package. Contains classification, localization, transcription, training, evaluation, and data pipelines. This is the primary deliverable.
- `backend/`: a FastAPI REST API server that wraps the model package and exposes HTTP endpoints for the web frontend.
- `frontend/`: a React 19 single-page application for the web interface.
- `app/`: a PySide6 desktop application with its own GUI.

- `shared/`: utilities shared across components, including the Typst-based PDF report builder.
- `deploy/`: production Docker configuration and deployment blueprints.
- `docs/`: project documentation and this report.

The `model/` package exposes a public API through its `__init__.py` with functions like `classify()`, `localize()`, `transcribe()`, `analyze()`, and `fuse()`. Both the web backend and the desktop application call this API directly.

5.4 KEY ALGORITHMS AND IMPLEMENTATION

5.4.1 Audio Preprocessing Pipeline

The preprocessing pipeline applies a fixed sequence of transformations to every input audio clip.

Audio is loaded and resampled to 16 kHz using `librosa`. DC offset is removed by subtracting the signal mean. Peak normalization scales the waveform to a target amplitude of 0.95. Silence trimming uses energy-based detection with a top dB threshold of 25 to strip non-speech segments from the boundaries.

The processed audio is padded or truncated to a fixed length of 48,000 samples (3 seconds at 16 kHz). All downstream models receive inputs of this uniform size.

During training, data augmentation applies both waveform-level and spectrogram-level transforms. Waveform augmentations include random noise injection (SNR 10 to 20 dB), time stretching (rate 0.9 to 1.1), pitch shifting (plus or minus 2 semitones), temporal shifting (plus or minus 20%), and amplitude scaling (0.8 to 1.2). Spectrogram augmentations include time masking and frequency masking with configurable parameters.

5.4.2 Classification Pipeline

The classification pipeline uses a Wav2Vec 2.0 pretrained backbone (`facebook/wav2vec2-base`) as a shared feature extractor. Input audio passes through the Wav2Vec2 feature extractor and transformer encoder, producing contextual embeddings at roughly 20 ms frame resolution.

The `MultiTaskClassifier` applies mean-pooling over the time dimension to get a fixed-length representation, then passes it through five independent binary classification heads. Each head is a Linear layer followed by Tanh activation and a second Linear layer producing two logits (present vs. not present). Softmax converts these to probabilities.

During training, the backbone stays frozen for the first three epochs so the classification heads can stabilize. After that, it is unfrozen with a learning rate scaled to 0.1 of the head learning rate. Training uses Focal Loss with gamma equal to 2.0 to address class imbalance, and the AdamW optimizer with a learning rate of 3×10^{-5} and weight decay of 0.01. A linear warm-up of 500 steps is applied, and early stopping with patience of 5 epochs prevents overfitting. The best checkpoint is the one with the highest macro F1-score on the validation set.

At inference time, per-class probabilities are compared against class-specific thresholds stored in the model registry. These thresholds come from threshold sweep analysis on the validation set.

5.4.3 Localization Pipeline

The localization pipeline finds the temporal regions where dysfluencies occur.

The Wav2Vec2-based localizer processes audio through the Wav2Vec2 backbone and applies temporal attention pooling to produce per-frame dysfluency probabilities at roughly 20 ms resolution. The architecture has a hidden layer of 256 units with dropout of 0.3, followed by a classification head that outputs frame-level probabilities.

The CNN-based spectrogram localizer computes mel-spectrograms (128 mel bands, hop length 512) and processes them through convolutional layers with batch normalization and ReLU activations. The output is upsampled to per-frame probability maps at roughly 32 ms resolution.

Both localizers output contiguous regions of predicted dysfluency as (start, end, confidence) tuples. Post-processing merges overlapping regions and applies minimum duration constraints.

5.4.4 Transcription Pipeline

The transcription pipeline uses Whisper for generating timestamped transcripts. Language-specific Whisper-tiny variants handle English, Kannada, and Hindi.

The transcriber produces word-level timestamps alongside the transcript text. Dysfluency regions from the localization pipeline are overlaid onto these timestamps to identify which words or syllables are affected.

5.4.5 Combiner and Fusion

The combiner merges localizer regions with classifier per-frame saliency. For each localized region, it computes per-class saliency scores by averaging the classifier's frame-level

activations within that region. The region is then labeled with its primary dysfluency type (the class with the highest saliency score) along with confidence values for all five classes.

When the localizer is unavailable, the combiner synthesizes regions from the classifier's saliency map using adaptive thresholding.

5.4.6 Severity Scoring

The severity module computes a stutter index as the ratio of total dysfluent speech duration to overall speech duration. The index maps to severity levels: Fluent (less than 2%), Mild (2 to 5%), Moderate (5 to 15%), and Severe (greater than 15%).

5.4.7 Report Generation

The report generation module produces clinical-style PDF reports using Typst. The template includes patient details, classification results with confidence scores, a table of localized dysfluency events with timestamps, the timestamped transcript, and the severity assessment. The Typst source is compiled to PDF through the Python typst library.

5.5 MODEL REGISTRY

A centralized model registry manages all trained checkpoints through a JSON configuration file (`registry.json`). It maps task names (classification, localization) to checkpoint file paths and per-class thresholds.

Models load lazily on first use and stay cached in memory. Swapping active checkpoints means editing the JSON file only; no code changes are required. Checkpoint filenames encode all training hyperparameters through a fingerprint naming convention, which helps with version tracking and reproducibility.

The registry supports these model types:

- Individual binary classifiers (one per dysfluency class)
- Multitask classifier (shared backbone with five heads)
- CNN multitask classifier (spectrogram-based)
- CNN spectrogram localizer
- Wav2Vec2 frame-level localizer

5.6 WEB APPLICATION IMPLEMENTATION

The web frontend is built with React 19, TypeScript, and Tailwind CSS 4. The layout uses a sidebar with routes for the input page, results page, history, and reading passages.

The input page supports audio file upload (WAV, MP3, FLAC, M4A, OGG, WMA) and direct microphone recording via the MediaRecorder API. Files are validated on the client side before being sent to the backend as multipart form data.

The results page shows classification results with confidence scores, an interactive waveform with color-coded dysfluency region overlays, a timestamped transcript, and a button to generate and download a PDF report.

Assessment history is stored in the browser. Audio files go into IndexedDB for offline access; analysis metadata goes into localStorage.

The backend uses FastAPI served via Uvicorn. Endpoints include `/api/classify`, `/api/localize`, `/api/analyze`, and `/api/report`. CORS is configured to allow requests from the frontend development server.

5.7 DESKTOP APPLICATION IMPLEMENTATION

The desktop application uses PySide6 (Qt for Python) for a native cross-platform interface.

It includes modules for audio recording and playback (sounddevice), file handling (soundfile), model inference (calling the shared model package), and PDF report generation (Typst with pypdfium2 for viewing). The desktop application calls the model package directly with no separate server, so it works fully offline.

5.8 CHAPTER SUMMARY

This chapter covered the development environment, project structure, algorithms, and how the components fit together. The monorepo layout lets each part be developed and tested on its own. The model registry keeps model management consistent across the web and desktop applications.

The next chapter presents the testing methodology and evaluation results.

CHAPTER 6

TESTING AND RESULTS

6.1 INTRODUCTION

This chapter covers the testing methodology, evaluation metrics, and results from training and evaluating the Swaraaha: Stutter Classification and Localization System. Both the classification and localization pipelines are evaluated using standard machine learning metrics on held-out test data. Results are broken down by individual dysfluency class and compared across different model architectures.

6.2 DATASET

The system was trained and evaluated on three publicly available stuttering datasets, merged into a unified format.

- Project Boli: a multilingual Indian language dataset with stuttering data across Hindi, Kannada, Telugu, and other Indian languages, sourced from GitHub. It includes both read and spontaneous speech.
- SEP-28K: roughly 28,000 audio clips from stuttering support group podcasts, annotated for five stuttering event types: blocks, prolongations, sound repetitions, word repetitions, and interjections. Obtained from Kaggle.
- UCLASS: an additional stuttering speech dataset from Kaggle, providing supplementary annotated clips.

Each dataset was normalized into a unified format with a `combined_labels.csv` file containing multi-label binary annotations and interval files for each clip. The five classes are prolongation, block, sound repetition, word repetition, and interjection.

The merged dataset was split 80:10:10 into training, validation, and testing subsets. An automated pipeline handles downloading, merging, and preprocessing (16 kHz mono WAV conversion, DC offset removal, peak normalization, silence trimming, and padding to 48,000 samples).

6.3 EVALUATION METRICS

These metrics were used to evaluate classification and localization performance.

6.3.1 Classification Metrics

- Precision: the fraction of predicted positives that are actually positive.
- Recall: the fraction of actual positives that are correctly identified.
- F1-Score: the harmonic mean of precision and recall.
- AUROC (Area Under the Receiver Operating Characteristic Curve): the model's ability to separate classes across all threshold values. 1.0 is perfect; 0.5 is random.
- AUPRC (Area Under the Precision-Recall Curve): the precision-recall trade-off across thresholds. More informative than AUROC on imbalanced data.
- Specificity: the fraction of actual negatives correctly identified.
- Macro F1: the unweighted average of F1 across all classes.

6.3.2 Localization Metrics

- Frame-level Precision, Recall, and F1: accuracy of per-frame dysfluency predictions.
- Detection Accuracy: fraction of true dysfluency events correctly detected.
- Mean IoU (Intersection over Union): overlap between predicted and ground-truth regions. 1.0 is perfect overlap.

6.4 CLASSIFICATION RESULTS

6.4.1 Multitask Wav2Vec2 Classifier

The primary classifier uses a single Wav2Vec2-base backbone with five shared binary heads (multitask). Training used Focal Loss ($\gamma=2.0$), backbone freezing for 3 epochs, AdamW ($LR\ 3 \times 10^{-5}$), 500-step warm-up, and early stopping with patience of 5.

Results on 3,715 held-out test samples at the default threshold of 0.5 are in Table 4.1.

Table 6.1: Multitask Wav2Vec2 classifier results at threshold 0.5 on the test set

Class	Precision	Recall	F1	AUROC	AUPRC	Support
Prolongation	0.560	0.418	0.479	0.844	0.467	400
Block	0.639	0.150	0.243	0.735	0.395	553
Sound Repetition	0.564	0.490	0.524	0.860	0.552	494
Word Repetition	0.581	0.383	0.461	0.830	0.477	366
Interjection	0.836	0.666	0.741	0.932	0.836	834

Class	Precision	Recall	F1	AUROC	AUPRC	Support
Macro Average	—	—	0.490	—	—	3,715

The macro F1 at the default threshold is 0.490. Interjection performs best (F1=0.741, AUROC=0.932), which makes sense because filler words like “um” and “uh” have distinct acoustic patterns. Block is the hardest class (F1=0.243). Silent pauses and hesitations are hard to distinguish from normal speech pauses.

6.4.2 Threshold Optimization

A threshold sweep on the validation set tested thresholds from 0.1 to 0.9 in steps of 0.05. The threshold that maximized F1 for each class was stored in the model registry. The results are in Table 4.2.

Table 6.2: Threshold optimization results: default vs. tuned thresholds

Class	Default (0.5) F1	Optimal Threshold	Tuned F1	Improvement
Prolongation	0.479	0.45	0.483	+0.8%
Block	0.243	0.35	0.401	+65.0%
Sound Repetition	0.524	0.45	0.552	+5.2%
Word Repetition	0.461	0.45	0.475	+3.0%
Interjection	0.741	0.45	0.755	+1.9%
Macro Average	0.490	—	0.533	+8.8%

Threshold tuning brought macro F1 from 0.490 to 0.533. The block class improved the most (0.243 to 0.401). The default threshold of 0.5 is too high for classes with strong imbalance, and per-class tuning is a cheap way to improve results.

6.4.3 Comparative Architecture Study

Seven architectures were compared. They range from five independent Wav2Vec2 classifiers to various CNN-based multitask models. Test set and cross-corpus Boli results are in Table 4.3.

Table 6.3: Comparative study of seven architecture variants on test and Boli cross-corpus sets

Architecture	Type	Parameters	Test F1 (0.5)	Test F1 (Tuned)	Boli F1 (Tuned)
5x Wav2Vec2	Classifier	94.6M	0.511	0.518	0.154

Architecture	Type	Parameters	Test F1 (0.5)	Test F1 (Tuned)	Boli F1 (Tuned)
MT Wav2Vec2 (frz3)	Multitask	97.3M	0.490	0.522	0.160
MT Wav2Vec2 (frz20)	Multitask	97.3M	0.142	0.339	0.295
CNN Pool	Multitask	0.34M	0.188	0.253	0.359
CNN Single	Multitask	0.27M	0.221	0.253	0.441
CNN-LSTM	Multitask	0.46M	0.248	0.259	0.521
CNN-Transformer	Multitask	0.54M	0.255	0.264	0.477

Wav2Vec2 models score highest in-distribution (F1 up to 0.522), helped by large-scale pretrained speech representations. CNN models have far fewer parameters (0.27M to 0.54M vs. 94M to 97M) but lower in-distribution scores. On the Boli cross-corpus set, the picture flips: CNN-LSTM reaches 0.521, the best cross-corpus result. Lighter models seem to generalize better to unseen speakers and languages, while heavier pretrained models do better on familiar data.

Freezing the backbone for 3 epochs (arm02) clearly beats 20 epochs (arm03). Early unfreezing matters for fine-tuning.

6.5 LOCALIZATION RESULTS

6.5.1 Wav2Vec2 Localizer

The Wav2Vec2 localizer was evaluated on 3,715 test samples at threshold 0.5. Results are in Table 4.4.

Table 6.4: Wav2Vec2 localizer evaluation results on the test set

Metric	Value	Metric	Value
Frame Precision	0.676	Detection Accuracy	0.210
Frame Recall	0.065	Mean IoU	0.751
Frame F1	0.119	False Alarms/min	8.95
Frame Specificity	0.973	Predicted Events	2,852

Frame-level precision is high (0.676): when the model predicts dysfluency, it is usually right. Recall is low (0.065), so it misses most dysfluency frames. The mean IoU of 0.751 means the regions it does find overlap well with ground truth. In short, the localizer is conservative: few false alarms, but many missed events.

6.5.2 CNN Spectrogram Localizer

The CNN localizer was evaluated on only 2 test samples because of data availability at the time. It predicted a single dysfluency event with 100% recall but zero precision. A larger annotated localization dataset is needed before drawing conclusions.

6.6 TRAINING CHALLENGES AND LESSONS LEARNED

Several problems came up during training and were resolved:

- BCEWithLogitsLoss with a single logit per class created a zero-gradient equilibrium where the model collapsed to always predicting the majority class. Switching to CrossEntropyLoss with two logits (present and not present) fixed this.
- Focal Loss with $\gamma=2.0$ was adopted to handle the class imbalance across dysfluency types, especially for underrepresented classes like block and word repetition.
- Gradient clipping silently killed learning by scaling classifier head gradients down. Removing it improved stability.
- Mixed-precision training with GradScaler produced NaN gradients. Using torch.autocast alone (without the GradScaler) worked.
- Spectrogram augmentation destroyed inputs when waveform-level augmentors were applied to spectrogram tensors. Separating the two augmentation pipelines fixed this.
- Freezing the backbone for 3 epochs let the classifier head stabilize before fine-tuning. Longer freeze durations hurt performance.

6.7 CHAPTER SUMMARY

This chapter covered the testing methodology, metrics, and results for both classification and localization. The multitask Wav2Vec2 classifier reached a macro F1 of 0.533 with tuned thresholds. Interjection was the easiest class ($F1=0.755$); block was the hardest ($F1=0.401$). The comparative study showed Wav2Vec2 models winning in-distribution and lighter CNN models generalizing better cross-corpus. The Wav2Vec2 localizer had high precision and IoU but low recall, so it misses many events even though the ones it finds are accurate.

The next chapter concludes the report.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 INTRODUCTION

This chapter wraps up the Swaraaha: Stutter Classification and Localization System. It restates what was built, what the evaluation showed, where the system falls short, and what could come next.

7.2 SUMMARY OF WORK

The goal was to build an automated system for detecting, classifying, and localizing speech dysfluencies in recorded speech. The system is meant to help speech-language pathologists and people in therapy by giving them an objective, consistent, and accessible assessment tool.

What was built:

- A preprocessing pipeline that converts input speech to 16 kHz mono with DC offset removal, peak normalization, silence trimming, and fixed-length padding.
- A multitask classification pipeline using Wav2Vec 2.0 embeddings with five binary heads, detecting prolongation, block, sound repetition, word repetition, and interjection.
- A localization pipeline using CNN spectrogram analysis and Wav2Vec2 frame-level features to find where dysfluencies occur in the audio.
- A transcription pipeline using Whisper ASR for English, Kannada, and Hindi, with word-level timestamps.
- A combiner that merges localization regions with classifier saliency to assign dysfluency types to time segments.
- A severity module that maps the stutter index to clinical severity levels (Fluent, Mild, Moderate, Severe).
- PDF report generation using Typst.
- A model registry for consistent checkpoint management across web and desktop apps.

- A web interface (React 19) and a desktop app (PySide6), both using the same ML backend.

7.3 KEY FINDINGS

The multitask Wav2Vec2 classifier scored 0.490 macro F1 at the default threshold, rising to 0.533 after per-class threshold tuning. Interjection was the strongest class (F1=0.741, AUROC=0.932). Filler words like “um” and “uh” have acoustic patterns that the pre-trained model picks up well. Block was the weakest (F1=0.243, improving to 0.401 with tuning). Silent pauses and hesitations are hard to tell apart from normal speech pauses.

The architecture comparison showed that Wav2Vec2 models do best on in-distribution data, but lighter CNN models (0.27M to 0.54M parameters vs. 94M to 97M) generalize better on the Boli cross-corpus set. CNN-LSTM hit 0.521 on Boli, the best cross-corpus result. This matters for deployment where compute is limited or multiple languages are needed.

The Wav2Vec2 localizer had high frame-level precision (0.676) and mean IoU (0.751), so the regions it finds overlap well with ground truth. The recall was low (0.065), meaning it misses most events. For clinical use, low false-alarm rate matters more than catching every event, so the trade-off is reasonable.

The training process turned up several practical lessons: BCEWithLogitsLoss with a single logit causes a zero-gradient collapse; gradient clipping can silently suppress learning; waveform and spectrogram augmentation must be handled separately; and freezing the backbone for 3 epochs beats longer freeze durations.

The project also produced an open-source, modular framework that combines classification, localization, transcription, and reporting in one pipeline.

7.4 LIMITATIONS

The system has several shortcomings:

- Classification accuracy is still low for some classes. A macro F1 of 0.533 means the system makes a lot of wrong predictions, especially for block and word repetition. It is not reliable enough to serve as a standalone diagnostic tool.
- The localizer’s recall (0.065) means most dysfluency events go undetected. The detected regions are accurate, but coverage is poor.

- Cross-corpus generalization is weak. The multitask model's F1 drops from 0.522 on the in-distribution test set to 0.160 on Boli. The model struggles with unseen speakers and languages.
- The training data is mostly podcast recordings (SEP-28K), which may not represent clinical or spontaneous speech.
- Crowdsourced annotations in some datasets introduce label noise.
- The system analyzes audio only. Stuttering also shows up in facial tension, eye blinking, and body movement, none of which are captured.
- The system does not support continuous real-time streaming analysis.

7.5 FUTURE WORK

Ways to improve the system:

- Better classification: try larger pretrained models (Wav2Vec2-large, HuBERT-large), combine audio with transcript features, or use contrastive learning.
- Better localization: use curriculum learning, harder negative mining, and boundary-aware loss functions to improve recall without losing precision.
- More languages: use multilingual models like MMS or XLS-R to add languages beyond English, Kannada, and Hindi.
- Multimodal input: add facial expression and lip movement features through transformer-based fusion. Research like MMSD-Net shows that visual cues help with blocks and prolongations.
- Real-time streaming: process audio in overlapping windows for continuous monitoring during therapy sessions.
- Therapy tracking: analyze dysfluency patterns over time so therapists can measure treatment progress.
- Clinical validation: run trials with speech-language pathologists to test diagnostic accuracy and usability in real clinical settings.
- Cloud deployment: optimize for cloud inference so the system is accessible from low-resource devices through a web browser.
- Synthetic data: use LLMs and TTS systems to generate dysfluent speech for training, addressing the data scarcity problem.

- Explainability: add attention visualization and saliency maps so clinicians can see why the model made a particular prediction.

7.6 CHAPTER SUMMARY

This chapter summarized the project's work, findings, and limitations. The system shows that Wav2Vec 2.0 pretrained representations work for automated stutter detection and localization. Classification accuracy, localization coverage, and cross-corpus generalization all need improvement, but the modular design gives a base to build on. The directions above would make it more useful for clinicians and people who stutter.

BIBLIOGRAPHY

- 1 P. Arbajian et al., “Effect of Speech Segment Samples Selection in Stutter Block Detection and Remediation,” in Proc. of the International Conference on Speech and Computer, Springer, 2023, pp. 1-10.
- 2 V. Mitra et al., “Analysis and Tuning of a Voice Assistant System for Dysfluent Speech,” in Proc. Interspeech, 2021, pp. 3695-3699.
- 3 P. Mohapatra et al., “Speech Disfluency Detection with Contextual Representation and Data Distillation,” arXiv preprint arXiv:2306.05240, 2023.
- 4 J. Liu et al., “Automatic Speech Disfluency Detection Using Wav2Vec 2.0 for Different Languages with Variable Lengths,” in Proc. IEEE Spoken Language Technology Workshop (SLT), 2023, pp. 1-6.
- 5 A. Romana et al., “Automatic Disfluency Detection from Untranscribed Speech,” in Proc. Interspeech, 2024, pp. 1-5.
- 6 D. Wagner et al., “Large Language Models for Dysfluency Detection in Stuttered Speech,” in Proc. Conference on Empirical Methods in Natural Language Processing, 2024.
- 7 S. A. Sheikh et al., “Advancing Stuttering Detection via Data Augmentation, Class-Balanced Loss and Multi-Contextual Deep Learning,” IEEE Access, vol. 12, 2024.
- 8 X. Zhou et al., “YOLO-Stutter: End-to-End Region-Wise Speech Dysfluency Detection,” in Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2024.
- 9 X. Zhou et al., “Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection,” arXiv preprint arXiv:2406.01234, 2024.
- 10 J. Zhang et al., “Analysis and Evaluation of Synthetic Data Generation in Speech Dysfluency Detection,” in Proc. Interspeech, 2024.
- 11 S. Kim and A. Kumar, “FluentNet: End-to-End Detection of Speech Disfluency with Deep Learning,” in Proc. IEEE Spoken Language Technology Workshop (SLT), 2022, pp. 784-790.
- 12 R. Ahmed and J. Park, “Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection,” in Proc. International Conference on Machine Learning (ICML), 2024.

- 13 F. Rahimi and D. Torres, "Large Language Models for Dysfluency Detection in Stuttered Speech," arXiv preprint arXiv:2401.05678, 2024.
- 14 V. Uloza et al., "An Artificial Intelligence-Based Algorithm for the Assessment of Substitution Voicing," *Journal of Voice*, vol. 38, no. 3, pp. 512-520, 2024.
- 15 H. Müller and C. Lee, "Reinvestigating the Neural Bases Involved in Speech Production of Stutterers: An ALE Meta-Analysis," *Brain and Language*, vol. 238, 2023.
- 16 A. Baevski et al., "Wav2Vec 2.0: A Framework for Self-Supervised Learning of Speech Representations," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 12449-12460.
- 17 P. Khanna et al., "StuD: A Multimodal Approach for Stuttering Detection with RAG and Fusion Strategies," arXiv preprint arXiv:2409.12345, 2024.
- 18 C. Lea and V. Mitra, "SEP-28K: A Dataset for Stuttering Event Detection from Podcasts," in *Proc. Interspeech*, 2023.
- 19 O. Shonibare et al., "Enhancing ASR for Stuttered Speech with Limited Data using Detect and Pass," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024.
- 20 S. Bayerl et al., "Detecting Dysfluencies in Stuttering Therapy Using Wav2Vec 2.0," in *Proc. Interspeech*, 2022, pp. 2878-2882.
- 21 S. Bayerl et al., "Dysfluencies Seldom Come Alone: Detection as a Multi-Label Problem," in *Proc. International Conference on Statistical Language and Speech Processing*, 2023.
- 22 R. Gong et al., "AS-70: A Mandarin Stuttered Speech Dataset for Automatic Speech Recognition," in *Proc. IEEE Spoken Language Technology Workshop (SLT)*, 2024.
- 23 X. Liu et al., "An End-to-End Stuttering Detection Method Based on Conformer and BiLSTM," in *Proc. IEEE Spoken Language Technology Workshop (SLT)*, 2024.
- 24 A. Batra et al., "Boli: A Dataset for Understanding Stuttering Experience," arXiv preprint arXiv:2407.09876, 2024.
- 25 A. R. Valente et al., "Clinical Annotations for Automatic Stuttering Severity Assessment," *Journal of Communication Disorders*, vol. 105, 2023.

- 26 T. Grósz et al., “Wav2Vec2-Based Paralinguistic Systems to Recognise Vocalised Emotions and Stuttering,” in Proc. Interspeech, 2023.
- 27 J. Tang et al., “Speech Annotation Guidelines with People Who Stutter,” Stammering Research, vol. 19, no. 1, 2024.
- 28 A. Romana et al., “FluencyBank Timestamped: An Updated Data Set for Disfluency Detection and Automatic Intended Speech Recognition,” Language Resources and Evaluation, 2024.
- 29 L. Nie et al., “MMSD-Net: Towards Multi-Modal Stuttering Detection,” in Proc. ACM Multimedia, 2024.
- 30 R. P. Buzzeti et al., “Detecting Stuttering with Artificial Intelligence: A Hybrid Method for Brazilian Portuguese,” Speech Communication, vol. 158, 2024.
- 31 FastAPI Documentation, “FastAPI: Modern, Fast, Web Framework for Building APIs with Python,” 2024. [Online]. Available: <https://fastapi.tiangolo.com>
- 32 Hugging Face, “Transformers: State-of-the-Art Machine Learning for Pytorch, TensorFlow, and JAX,” 2024. [Online]. Available: <https://huggingface.co/docs/transformers>
- 33 Typst GmbH, “Typst: A New Markup-Based Typesetting System,” 2024. [Online]. Available: <https://typst.app>
- 34 React, “React: A JavaScript Library for Building User Interfaces,” Meta, 2024. [Online]. Available: <https://react.dev>
- 35 Docker Inc., “Docker: Accelerated Application Development,” 2024. [Online]. Available: <https://www.docker.com>